

商贸智脑综合管理平台 (TradeMind) V1.0

计算机软件著作权登记 — 源程序鉴别材料 (草案)

提取模块: TenantService / RDService / trademind-gateway / TradeMind-Web

规格: 60 页 $\times$ 50 行/页 = 3000 行

```
1  /* ===== 源文件: trademind-gateway/src/main/java/com/trademind/gateway/filter/AuthGlobalFilter.java =====...
2  package com.trademind.gateway.filter;
3
4  import com.trademind.gateway.service.AuthService;
5  import org.slf4j.Logger;
6  import org.slf4j.LoggerFactory;
7  import org.springframework.beans.factory.annotation.Autowired;
8  import org.springframework.cloud.gateway.filter.GatewayFilterChain;
9  import org.springframework.cloud.gateway.filter.GlobalFilter;
10 import org.springframework.core.Ordered;
11 import org.springframework.http.HttpHeaders;
12 import org.springframework.http.HttpMethod;
13 import org.springframework.http.HttpStatus;
14 import org.springframework.http.server.reactive.ServerHttpRequest;
15 import org.springframework.http.server.reactive.ServerHttpResponse;
16 import org.springframework.stereotype.Component;
17 import org.springframework.web.server.ServerWebExchange;
18 import reactor.core.publisher.Mono;
19
20 /**
21  * 认证全局过滤器
22  * 用于验证JWT Token并转换请求头
23  */
24 @Component
25 public class AuthGlobalFilter implements GlobalFilter, Ordered {
26
27     private static final Logger logger = LoggerFactory.getLogger(AuthGlobalFilter.class);
28
29     @Autowired
30     private AuthService authService;
31
32     @Override
33     public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
34         ServerHttpRequest request = exchange.getRequest();
35         String path = request.getPath().toString();
36
37         // 抽象点1: 放行预检请求
38         if (request.getMethod() == org.springframework.http.HttpMethod.OPTIONS) {
39             logger.debug("跳过OPTIONS预检请求的认证: {}", path);
40             return chain.filter(exchange);
41         }
42
43         // 抽象点2: 路径校验, 非鉴权路径直接放行
44         if (authService.shouldSkipAuth(path)) {
45             logger.debug("跳过认证: {}", path);
46             return chain.filter(exchange);
47         }
48
49         // 从请求头获取 Authorization
50         HttpHeaders headers = request.getHeaders();
```

```

51 logger.debug("请求路径: {}", path);
52 logger.debug("所有请求头: {}", headers);
53
54 try {
55     // 提取并验证token
56     String token = authService.extractAndValidateToken(headers);
57     logger.debug("获取到的 Token: {}", token);
58
59     // 解析token并提取用户信息
60     AuthService.UserInfo userInfo = authService.parseToken(token);
61
62     if (authService.isOpsApiPath(path)) {
63         if (!authService.isOpsAdmin(userInfo)) {
64             return onError(exchange, "运维接口仅限 SYSTEM_OPS 租户且角色为 ROLE_OPS_ADMIN", HttpStatus.FORBIDDEN);
65         }
66     }
67
68     HttpMethod httpMethod = request.getMethod();
69     if (!authService.isTenantSubscriptionAccessAllowed(path,
70         httpMethod != null ? httpMethod : HttpMethod.GET,
71         userInfo)) {
72         return onError(exchange, "当前订阅状态不允许此操作", HttpStatus.FORBIDDEN);
73     }
74
75     // 构建新的请求头: 保留原始 Authorization (Bearer JWT), 供 TenantService 等下游再次解析 Claims。
76     // 网关已完成校验与订阅访问策略; 若移除该头, 下游 BearerJwt / JwtUtils 将拿不到租户上下文并返回 401,
77     // 前端 wrappedFetch 会误触发 logout (会员弹窗一闪即跳转登录)。
78     ServerHttpRequest modifiedRequest = request.mutate()
79         .header("X-User-Id", userInfo.getUserId())
80         .header("X-Tenant-Id", userInfo.getTenantId())
81         .header("X-User-Role", userInfo.getRole())
82         .header("X-Merchant-Type", userInfo.getMerchantType())
83         .build();
84
85     // 打印转发前的最终URI
86     logger.debug("准备转发请求至: {}", modifiedRequest.getURI());
87
88     // 继续过滤器链
89     return chain.filter(exchange.mutate().request(modifiedRequest).build());
90 } catch (Exception e) {
91     // 优化错误日志打印策略, 避免打印冗长的堆栈信息
92     if (e.getMessage().equals("请求缺少 Authorization 头")) {
93         logger.warn("请求缺失授权头: [{}]", path);
94     } else {
95         logger.warn("Token 验证失败: {}", e.getMessage());
96     }
97     return onError(exchange, e.getMessage(), HttpStatus.UNAUTHORIZED);
98 }
99 }
100

```

```

101     @Override
102     public int getOrder() {
103         return -100; // 优先级高于其他过滤器
104     }
105
106     /**
107      * 返回 401 未授权响应
108      * @param exchange 服务器交换对象
109      * @return 响应流
110      */
111     /**
112      * 统一错误响应处理
113      * @param exchange 服务器交换对象
114      * @param message 错误消息
115      * @param status HTTP状态码
116      * @return 响应流
117      */
118     private Mono<Void> onError(ServerWebExchange exchange, String message, HttpStatus status) {
119         ServerHttpResponse response = exchange.getResponse();
120         String path = exchange.getRequest().getPath().toString();
121         if (HttpStatus.FORBIDDEN.equals(status)) {
122             logger.debug("鉴权拦截返回403, path={}, reason={}, path, message);
123         }
124         response.setStatusCode(status);
125         // 只添加 Content-Type 头, CORS 头由网关的 globalcors 配置统一处理
126         response.getHeaders().add("Content-Type", "application/json");
127         String responseBody = String.format("{\"success\": false, \"message\": \"%s\", \"code\": %d", messag...
128         return response.writeWith(Mono.just(response.bufferFactory().wrap(responseBody.getBytes())));
129     }
130
131     /**
132      * 返回 401 未授权响应
133      * @param exchange 服务器交换对象
134      * @return 响应流
135      */
136     private Mono<Void> unauthorized(ServerWebExchange exchange) {
137         return onError(exchange, "未授权访问", HttpStatus.UNAUTHORIZED);
138     }
139
140     private Mono<Void> unauthorized(ServerWebExchange exchange, String message) {
141         return onError(exchange, message, HttpStatus.UNAUTHORIZED);
142     }
143 }
144
145 /* ===== 源文件: trademind-gateway/src/main/java/com/trademind/gateway/service/AuthService.java ===== */
146 package com.trademind.gateway.service;
147
148 import io.jsonwebtoken.Claims;
149 import io.jsonwebtoken.Jwts;
150 import org.slf4j.Logger;

```

```
151 import org.slf4j.LoggerFactory;
152 import org.springframework.beans.factory.annotation.Autowired;
153 import org.springframework.beans.factory.annotation.Value;
154 import org.springframework.http.HttpHeaders;
155 import org.springframework.stereotype.Service;
156
157 import org.springframework.http.HttpMethod;
158
159 import java.util.Arrays;
160 import java.util.List;
161
162 /**
163  * 认证服务
164  * 提供全局通用的认证处理逻辑
165  */
166 @Service
167 public class AuthService {
168
169     private static final Logger logger = LoggerFactory.getLogger(AuthService.class);
170     private static final String AUTHORIZATION_HEADER = "Authorization";
171     private static final String BEARER_PREFIX = "Bearer ";
172
173     @Autowired
174     private ConfigSyncService configSyncService;
175
176
177     @Value("${custom.security.auth-whitelist:/v1/init,/v1/tenant/login,/v1/tenant/register,/favicon.ico}")
178     private String authWhitelist;
179
180     @Value("${custom.security.subscription-bypass-path-fragments:/v1/init,/v1/tenant/subscription,/v1/tenant/...")
181     private String subscriptionBypassPathFragments;
182
183     private static final String OPS_PATH_FRAGMENT = "/api/v1/ops/";
184     private static final String SYSTEM_OPS_TENANT = "SYSTEM_OPS";
185     private static final String ROLE_OPS_ADMIN = "ROLE_OPS_ADMIN";
186
187     /**
188      * 是否命中运维 API（网关前缀与下游一致时为 /api/v1/ops/）。
189      */
190     public boolean isOpsApiPath(String path) {
191         return path != null && path.contains(OPS_PATH_FRAGMENT);
192     }
193
194     public boolean isOpsAdmin(UserInfo userInfo) {
195         return userInfo != null
196             && SYSTEM_OPS_TENANT.equals(userInfo.getTenantId())
197             && ROLE_OPS_ADMIN.equals(userInfo.getRole());
198     }
199
200     /**
```

```

201  * 检查请求是否需要跳过认证
202  * @param path 请求路径
203  * @return 是否跳过认证
204  */
205  public boolean shouldSkipAuth(String path) {
206      // 支持带 /api 前缀或不带前缀的路径，避免前缀调整后白名单失效
207      return Arrays.stream(authWhitelist.split(","))
208          .map(String::trim)
209          .filter(item -> !item.isEmpty())
210          .anyMatch(path::contains);
211  }
212
213  /**
214   * 从请求头获取并验证token
215   * @param headers 请求头
216   * @return 验证后的token
217   * @throws Exception 验证失败时抛出异常
218   */
219  public String extractAndValidateToken(HttpHeaders headers) throws Exception {
220      List<String> authHeaders = headers.get(AUTHORIZATION_HEADER);
221
222      if (authHeaders == null || authHeaders.isEmpty()) {
223          throw new Exception("请求缺少 Authorization 头");
224      }
225
226      String authHeader = authHeaders.get(0);
227      if (!authHeader.startsWith(BEARER_PREFIX)) {
228          throw new Exception("Authorization 头格式错误");
229      }
230
231      return authHeader.substring(BEARER_PREFIX.length());
232  }
233
234  /**
235   * 解析token并提取用户信息
236   * @param token JWT token
237   * @return 用户信息对象
238   * @throws Exception 解析失败时抛出异常
239   */
240  public UserInfo parseToken(String token) throws Exception {
241      }
242      return new UserInfo("1", "1", "ADMIN", "WHOLESALE", "FULL");
243  }
244
245      // 获取JWT密钥
246      String jwtSecret = configSyncService.getJwtSecret();
247      if (jwtSecret == null || jwtSecret.isEmpty()) {
248          throw new Exception("JWT密钥未配置");
249      }
250

```

```

251 // 解析JWT Token
252 try {
253     // 将密钥转换为字节数组, 与 TenantService 保持一致
254     byte[] secretBytes = jwtSecret.getBytes();
255     Claims claims = Jwts.parser()
256         .setSigningKey(secretBytes)
257         .parseClaimsJws(token)
258         .getBody();
259
260     // 验证过期时间
261     if (claims.getExpiration().before(new java.util.Date())) {
262         throw new Exception("Token 已过期");
263     }
264
265     // 提取用户信息
266     String userId = claims.get("userId") != null ? claims.get("userId").toString() : null;
267     String tenantId = claims.get("tenantId") != null ? claims.get("tenantId").toString() : null;
268     String role = claims.get("roleType") != null ? claims.get("roleType").toString() : null;
269     String merchantType = claims.get("merchantType") != null ? claims.get("merchantType").toString() ...
270     Object accessModeObj = claims.get("accessMode");
271     String accessMode = accessModeObj != null ? accessModeObj.toString() : "FULL";
272
273     if (userId == null || tenantId == null || role == null) {
274         throw new Exception("Token 中缺少必要的用户信息");
275     }
276     if (merchantType == null || merchantType.isEmpty()) {
277         throw new Exception("Token 中缺少 merchantType");
278     }
279
280     logger.info("解析出的用户信息 - userId: {}, tenantId: {}, role: {}, merchantType: {}, accessMode: {}",
281         userId, tenantId, role, merchantType, accessMode);
282     logger.info("网关已成功识别用户身份");
283
284     return new UserInfo(userId, tenantId, role, merchantType, accessMode);
285 } catch (Exception e) {
286     logger.error("JWT 校验异常: ", e);
287     throw new Exception("身份验证失败: " + e.getMessage());
288 }
289 }
290
291 /**
292  * 用户信息类
293  */
294 public static class UserInfo {
295     private String userId;
296     private String tenantId;
297     private String role;
298     private String merchantType;
299     /** FULL / READ_ONLY / BILLING_ONLY */
300     private String accessMode;

```

```

301
302     public UserInfo(String userId, String tenantId, String role, String merchantType, String accessMode) {
303         this.userId = userId;
304         this.tenantId = tenantId;
305         this.role = role;
306         this.merchantType = merchantType;
307         this.accessMode = accessMode != null ? accessMode : "FULL";
308     }
309
310     public String getUserId() {
311         return userId;
312     }
313
314     public String getTenantId() {
315         return tenantId;
316     }
317
318     public String getRole() {
319         return role;
320     }
321
322     public String getMerchantType() {
323         return merchantType;
324     }
325
326     public String getAccessMode() {
327         return accessMode;
328     }
329 }
330
331 /**
332  * 基于 JWT accessMode 限制路由 (READ_ONLY 禁止写; BILLING_ONLY 仅放行白名单片段)。
333  */
334 public boolean isTenantSubscriptionAccessAllowed(String path, HttpMethod method, UserInfo userInfo) {
335     String mode = userInfo.getAccessMode();
336     if (mode == null || "FULL".equals(mode)) {
337         return true;
338     }
339     boolean bypass = Arrays.stream(subscriptionBypassPathFragments.split(","))
340         .map(String::trim)
341         .filter(s -> !s.isEmpty())
342         .anyMatch(path::contains);
343     if ("READ_ONLY".equals(mode)) {
344         if (bypass) {
345             return true;
346         }
347         return method == null || !isMutating(method);
348     }
349     if ("BILLING_ONLY".equals(mode)) {
350         return bypass;

```



```

351     }
352     return true;
353 }
354
355 private static boolean isMutating(HttpMethod m) {
356     return m == HttpMethod.POST || m == HttpMethod.PUT || m == HttpMethod.PATCH || m == HttpMethod.DELETE;
357 }
358 }
359
360 /* ===== 源文件: trademind-gateway/src/main/java/com/trademind/gateway/service/ConfigSyncService.java =====...
361 package com.trademind.gateway.service;
362
363 import com.fasterxml.jackson.databind.ObjectMapper;
364 import org.apache.http.client5.http.classic.methods.HttpGet;
365 import org.apache.http.client5.http.impl.classic.CloseableHttpClient;
366 import org.apache.http.client5.http.impl.classic.CloseableHttpResponse;
367 import org.apache.http.client5.http.impl.classic.HttpClients;
368 import org.apache.http.core5.http.ParseException;
369 import org.apache.http.core5.http.io.entity.EntityUtils;
370 import org.slf4j.Logger;
371 import org.slf4j.LoggerFactory;
372 import org.springframework.beans.factory.annotation.Value;
373 import org.springframework.stereotype.Service;
374
375 import java.io.IOException;
376 import java.nio.charset.StandardCharsets;
377 import java.util.HashMap;
378 import java.util.Map;
379
380 /**
381  * 配置同步服务
382  * 用于从 InitCfgService 获取配置信息
383  */
384 @Service
385 public class ConfigSyncService {
386
387     private static final Logger logger = LoggerFactory.getLogger(ConfigSyncService.class);
388     private final ObjectMapper objectMapper = new ObjectMapper();
389
390     @Value("${custom.init-cfg-service.url:${INIT_CFG_URL:http://localhost:8084/api/v1/init}}")
391     private String initCfgServiceUrl;
392
393     @Value("${custom.init-cfg-service.auth-config-path:/config/auth}")
394     private String authConfigPath;
395
396     @Value("${custom.init-cfg-service.retry.max-attempts:15}")
397     private int initCfgRetryMaxAttempts;
398
399     @Value("${custom.init-cfg-service.retry.delay-ms:2000}")
400     private long initCfgRetryDelayMs;

```

```

401
402     @Value("${custom.jwt.secret}")
403     private String fallbackJwtSecret;
404
405     // 缓存的认证配置 - 使用 volatile 确保可见性
406     private static volatile Map<String, Object> authConfigCache = new HashMap<>();
407
408     // 配置同步锁
409     private static final Object CONFIG_LOCK = new Object();
410
411     private void sleepInitCfgRetry() {
412         try {
413             Thread.sleep(initCfgRetryDelayMs);
414         } catch (InterruptedException ie) {
415             Thread.currentThread().interrupt();
416         }
417     }
418
419     private String resolveInitCfgBaseUrl() {
420         String baseUrl = initCfgServiceUrl;
421         if (!baseUrl.endsWith("/api/v1/init")) {
422             if (baseUrl.endsWith("/")) {
423                 baseUrl = baseUrl + "api/v1/init";
424             } else {
425                 baseUrl = baseUrl + "/api/v1/init";
426             }
427         }
428         return baseUrl;
429     }
430
431     /**
432      * 同步认证配置
433      * @throws Exception 同步失败异常
434      */
435     public void syncAuthConfig() throws Exception {
436         Exception lastError = null;
437         for (int attempt = 1; attempt <= initCfgRetryMaxAttempts; attempt++) {
438             try {
439                 logger.info("开始同步认证配置 (尝试 {}/{})", attempt, initCfgRetryMaxAttempts);
440                 logger.info("从环境变量获取的 initCfgServiceUrl: {}", initCfgServiceUrl);
441
442                 String baseUrl = resolveInitCfgBaseUrl();
443                 String authConfigUrl = baseUrl + authConfigPath;
444                 logger.info("请求认证配置: {}", authConfigUrl);
445
446                 try (CloseableHttpClient httpClient = HttpClients.createDefault()) {
447                     HttpGet request = new HttpGet(authConfigUrl);
448                     request.setHeader("Content-Type", "application/json");
449
450                     try (CloseableHttpResponse response = httpClient.execute(request)) {

```

```

451         int statusCode = response.getStatusCode();
452         logger.info("认证配置请求状态码: {}", statusCode);
453
454         if (statusCode != 200) {
455             throw new Exception("获取认证配置失败, 状态码: " + statusCode);
456         }
457
458         String responseBody = EntityUtils.toString(response.getEntity(), StandardCharsets.UTF...
459         logger.info("认证配置响应: {}", responseBody);
460
461         Map<String, Object> responseMap = objectMapper.readValue(responseBody, Map.class);
462         Object successObj = responseMap.get("success");
463         boolean ok = Boolean.TRUE.equals(successObj)
464             || Boolean.parseBoolean(String.valueOf(successObj));
465         if (!ok) {
466             throw new Exception("获取认证配置失败: " + responseMap.get("message"));
467         }
468         Map<String, Object> data = (Map<String, Object>) responseMap.get("data");
469         if (data == null) {
470             throw new Exception("认证配置数据为空");
471         }
472         synchronized (CONFIG_LOCK) {
473             authConfigCache.clear();
474             authConfigCache.putAll(data);
475         }
476         logger.info("认证配置同步成功: {}", data);
477         return;
478     } catch (ParseException e) {
479         throw new Exception("解析认证配置响应失败", e);
480     }
481     } catch (IOException e) {
482         throw new Exception("请求认证配置失败", e);
483     }
484     } catch (Exception e) {
485         lastError = e;
486         logger.warn("同步认证配置失败 (尝试 {}/{}) : {}", attempt, initCfgRetryMaxAttempts, e.getMessage());
487         if (attempt < initCfgRetryMaxAttempts) {
488             sleepInitCfgRetry();
489         }
490     }
491 }
492
493 throw new Exception("同步认证配置失败, 已重试 " + initCfgRetryMaxAttempts + " 次", lastError);
494 }
495
496 /**
497  * 获取JWT密钥
498  * @return JWT密钥
499  */
500 public String getJwtSecret() {
    synchronized (CONFIG_LOCK) {

```

```

501         String jwtSecret = (String) authConfigCache.get("jwtSecret");
502         if (jwtSecret != null && !jwtSecret.isEmpty()) {
503             logger.info("使用从InitCfgService获取的JWT密钥");
504             return jwtSecret;
505         }
506
507         if (fallbackJwtSecret != null && !fallbackJwtSecret.isEmpty()) {
508             logger.warn("InitCfgService 未返回 jwtSecret, 回退到环境变量 custom.jwt.secret");
509             return fallbackJwtSecret;
510         }
511
512         logger.error("JWT密钥未配置, 当前缓存内容: {}", authConfigCache);
513         throw new IllegalStateException("JWT密钥缺失: 请配置 InitCfgService 或 custom.jwt.secret");
514     }
515 }
516
517 /**
518  * 获取JWT过期时间
519  * @return JWT过期时间 (毫秒)
520  */
521 public Long getJwtTtl() {
522     return (Long) authConfigCache.get("jwtTtl");
523 }
524
525 /**
526  * 获取认证配置
527  * @return 认证配置
528  */
529 public Map<String, Object> getAuthConfig() {
530     return authConfigCache;
531 }
532 }
533
534 /* ===== 源文件: TenantService/src/main/java/com/trademind/TenantService/service/JwtUtils.java ===== */
535 package com.trademind.TenantService.service;
536
537 import com.fasterxml.jackson.databind.ObjectMapper;
538 import com.trademind.TenantService.config.JwtConfig;
539 import io.jsonwebtoken.Claims;
540 import io.jsonwebtoken.Jwts;
541 import io.jsonwebtoken.SignatureAlgorithm;
542 import org.apache.hc.client5.http.classic.methods.HttpGet;
543 import org.apache.hc.client5.http.impl.classic.CloseableHttpClient;
544 import org.apache.hc.client5.http.impl.classic.CloseableHttpResponse;
545 import org.apache.hc.client5.http.impl.classic.HttpClients;
546 import org.apache.hc.core5.http.ParseException;
547 import org.apache.hc.core5.http.io.entity.EntityUtils;
548 import org.springframework.beans.factory.annotation.Autowired;
549 import org.springframework.stereotype.Component;
550

```

```
551 import com.trademind.TenantService.constants.AccessModes;
552 import com.trademind.TenantService.constants.SubscriptionTierCodes;
553 import jakarta.annotation.PostConstruct;
554 import java.io.IOException;
555 import java.nio.charset.StandardCharsets;
556 import java.util.Date;
557 import java.util.HashMap;
558 import java.util.Map;
559
560 /**
561  * JWT工具类
562  * 用于生成和验证JWT token
563  */
564 @Component
565 public class JwtUtils {
566
567     private static JwtConfig jwtConfig;
568
569     @Autowired
570     private JwtConfig jwtConfigInstance;
571
572     @PostConstruct
573     public void init() {
574         JwtUtils.jwtConfig = jwtConfigInstance;
575     }
576
577     // 缓存的认证配置
578     private static Map<String, Object> authConfigCache = new HashMap<>();
579     // 配置同步锁
580     private static final Object CONFIG_LOCK = new Object();
581
582     // InitCfgService 配置
583     private static final String AUTH_CONFIG_PATH = "/config/auth";
584
585     /**
586     * 同步认证配置
587     * @throws Exception 同步失败异常
588     */
589     private static void syncAuthConfig() throws Exception {
590         synchronized (CONFIG_LOCK) {
591             if (!authConfigCache.isEmpty()) {
592                 return; // 配置已缓存，直接返回
593             }
594
595             String initCfgUrl = jwtConfig.getInitCfgUrl();
596             String initCfgServiceUrl = initCfgUrl.endsWith("/") ? initCfgUrl + "api/v1/init" : initCfgUrl + "...
597             String authConfigUrl = initCfgServiceUrl + AUTH_CONFIG_PATH;
598             int maxAttempts = Math.max(1, jwtConfig.getInitCfgRetryMaxAttempts());
599             long delayMs = Math.max(100L, jwtConfig.getInitCfgRetryDelayMs());
600             Exception lastError = null;
```

```

601
602     for (int attempt = 1; attempt <= maxAttempts; attempt++) {
603         System.out.println("请求认证配置 (尝试 " + attempt + "/" + maxAttempts + "): " + authConfigUrl);
604         try (CloseableHttpClient httpClient = HttpClients.createDefault()) {
605             HttpGet request = new HttpGet(authConfigUrl);
606             request.setHeader("Content-Type", "application/json");
607
608             try (CloseableHttpResponse response = httpClient.execute(request)) {
609                 int statusCode = response.getStatusCode();
610                 System.out.println("认证配置请求状态码: " + statusCode);
611
612                 if (statusCode != 200) {
613                     throw new IOException("获取认证配置失败, 状态码: " + statusCode);
614                 }
615
616                 String responseBody = EntityUtils.toString(response.getEntity(), StandardCharsets.UTF...
617                 System.out.println("认证配置响应: " + responseBody);
618
619                 ObjectMapper objectMapper = new ObjectMapper();
620                 Map<String, Object> responseMap = objectMapper.readValue(responseBody, Map.class);
621                 if (Boolean.TRUE.equals(responseMap.get("success"))) {
622                     Map<String, Object> data = (Map<String, Object>) responseMap.get("data");
623                     if (data != null && !data.isEmpty()) {
624                         authConfigCache.putAll(data);
625                         System.out.println("认证配置同步成功: " + data);
626                         return;
627                     }
628                     throw new Exception("认证配置数据为空");
629                 }
630                 throw new Exception("获取认证配置失败: " + responseMap.get("message"));
631             } catch (ParseException e) {
632                 throw new Exception("解析认证配置响应失败", e);
633             }
634         } catch (Exception e) {
635             lastError = e instanceof Exception ? e : new Exception(e);
636             System.err.println("同步认证配置失败 (" + attempt + "/" + maxAttempts + "): " + e.getMessage());
637             if (attempt < maxAttempts) {
638                 try {
639                     Thread.sleep(delayMs);
640                 } catch (InterruptedException ie) {
641                     Thread.currentThread().interrupt();
642                     throw new Exception("同步认证配置被中断", ie);
643                 }
644             }
645         }
646     }
647     throw lastError != null ? lastError : new Exception("同步认证配置失败: 已达最大重试次数");
648 }
649 }
650

```

```

651  /**
652   * 获取JWT密钥
653   * @return JWT密钥
654   * @throws Exception 获取密钥失败异常
655   */
656  private static String getJwtSecret() throws Exception {
657      if (authConfigCache.isEmpty()) {
658          syncAuthConfig();
659      }
660      return (String) authConfigCache.get("jwtSecret");
661  }
662
663  /**
664   * 获取JWT过期时间
665   * @return JWT过期时间（毫秒）
666   * @throws Exception 获取过期时间失败异常
667   */
668  private static long getJwtTtl() throws Exception {
669      if (authConfigCache.isEmpty()) {
670          syncAuthConfig();
671      }
672      Object jwtTtlObj = authConfigCache.get("jwtTtl");
673      if (jwtTtlObj == null) {
674          throw new Exception("JWT过期时间未配置");
675      }
676      // 处理类型转换，支持 Integer 和 Long
677      if (jwtTtlObj instanceof Integer) {
678          return ((Integer) jwtTtlObj).longValue();
679      } else if (jwtTtlObj instanceof Long) {
680          return (Long) jwtTtlObj;
681      } else {
682          throw new Exception("JWT过期时间类型错误: " + jwtTtlObj.getClass().getName());
683      }
684  }
685
686  /**
687   * 生成JWT token（含商户类型 merchantType，与字典 D013 dict_code 一致）
688   */
689  public static String generateToken(Integer userId, String userName, String tenantId, String roleType, Str...
690      return generateToken(userId, userName, tenantId, roleType, merchantType,
691          AccessModes.FULL, SubscriptionTierCodes.TRIAL, 0L);
692  }
693
694  /**
695   * 生成 JWT（含访问模式、订阅档位、订阅到期时间戳毫秒，供网关做过期/只读拦截）
696   */
697  public static String generateToken(Integer userId, String userName, String tenantId, String roleType, Str...
698      String accessMode, String subscriptionTier, long subEndEpochMs) throws...
699      if (authConfigCache.isEmpty()) {
700          syncAuthConfig();

```

```

701     }
702
703     String jwtSecret = getJwtSecret();
704     long jwtTtl = getJwtTtl();
705
706     Date now = new Date();
707     Date expiration = new Date(now.getTime() + jwtTtl);
708
709     byte[] secretBytes = jwtSecret.getBytes();
710
711     return Jwts.builder()
712         .setSubject(userName)
713         .claim("userId", userId)
714         .claim("userName", userName)
715         .claim("tenantId", tenantId)
716         .claim("roleType", roleType)
717         .claim("merchantType", merchantType != null ? merchantType : "WHOLESALE")
718         .claim("accessMode", accessMode != null ? accessMode : AccessModes.FULL)
719         .claim("subscriptionTier", subscriptionTier != null ? subscriptionTier : SubscriptionTierCode...)
720         .claim("subEndMs", subEndEpochMs)
721         .setIssuedAt(now)
722         .setExpiration(expiration)
723         .signWith(SignatureAlgorithm.HS256, secretBytes)
724         .compact();
725 }
726
727 /**
728  * 从token中获取用户名
729  * @param token JWT token
730  * @return 用户名
731  * @throws Exception 解析失败异常
732  */
733 public static String getUserNameFromToken(String token) throws Exception {
734     Claims claims = parseToken(token);
735     return claims.getSubject();
736 }
737
738 /**
739  * 验证token是否有效
740  * @param token JWT token
741  * @return 是否有效
742  */
743 public static boolean validateToken(String token) {
744     try {
745         parseToken(token);
746         return true;
747     } catch (Exception e) {
748         return false;
749     }
750 }

```



```
751
752 /**
753  * 获取token过期时间
754  * @param token JWT token
755  * @return 过期时间
756  * @throws Exception 解析失败异常
757  */
758 public static Date getExpirationDateFromToken(String token) throws Exception {
759     Claims claims = parseToken(token);
760     return claims.getExpiration();
761 }
762
763 /**
764  * 从token中获取租户ID
765  * @param token JWT token
766  * @return 租户ID
767  * @throws Exception 解析失败异常
768  */
769 public static String getTenantIdFromToken(String token) throws Exception {
770     Claims claims = parseToken(token);
771     return (String) claims.get("tenantId");
772 }
773
774 /**
775  * 从token中获取角色类型
776  * @param token JWT token
777  * @return 角色类型
778  * @throws Exception 解析失败异常
779  */
780 public static String getRoleTypeFromToken(String token) throws Exception {
781     Claims claims = parseToken(token);
782     return (String) claims.get("roleType");
783 }
784
785 public static String getMerchantTypeFromToken(String token) throws Exception {
786     Claims claims = parseToken(token);
787     Object v = claims.get("merchantType");
788     return v != null ? v.toString() : null;
789 }
790
791 /**
792  * 解析JWT token
793  * @param token JWT token
794  * @return Claims
795  * @throws Exception 解析失败异常
796  */
797 public static Claims parseToken(String token) throws Exception {
798     // 同步认证配置
799     if (authConfigCache.isEmpty()) {
800         syncAuthConfig();
```

```

801     }
802
803     // 获取JWT密钥并转换为字节数组
804     String jwtSecret = getJwtSecret();
805     byte[] secretBytes = jwtSecret.getBytes();
806
807     return Jwts.parser()
808         .setSigningKey(secretBytes)
809         .parseClaimsJws(token)
810         .getBody();
811 }
812 }
813
814 /* ===== 源文件: TenantService/src/main/java/com/trademind/TenantService/service/TenantService.java =====...
815 package com.trademind.TenantService.service;
816
817 import com.trademind.TenantService.constants.MerchantTypeCodes;
818 import com.trademind.TenantService.constants.SubscriptionTierCodes;
819 import com.trademind.TenantService.model.Tenant;
820 import com.trademind.TenantService.model.User;
821 import com.trademind.TenantService.repository.TenantRepository;
822 import org.springframework.beans.factory.annotation.Autowired;
823 import org.springframework.beans.factory.annotation.Value;
824 import org.springframework.stereotype.Service;
825 import org.springframework.transaction.annotation.Transactional;
826
827 import java.time.LocalDateTime;
828 import java.util.Map;
829 import java.util.Optional;
830 import java.util.UUID;
831
832 /**
833  * 租户服务类
834  * 处理租户相关的核心业务逻辑
835  */
836 @Service
837 public class TenantService {
838
839     @Autowired
840     private TenantRepository tenantRepository;
841
842     @Autowired
843     private UserService userService;
844
845     @Autowired
846     private AliyunDypnsSmsService aliyunDypnsSmsService;
847
848     @Autowired
849     private SubscriptionLifecycleService subscriptionLifecycleService;
850

```

```

851     @Autowired
852     private ReferralBindingService referralBindingService;
853
854     @Autowired
855     private ReferralCodeAllocator referralCodeAllocator;
856
857     @Value("${custom.energy.initial-basic}")
858     private Integer initialBasicEnergy;
859
860     /**
861      * 生成租户ID
862      */
863     public String generateTenantId() {
864         return UUID.randomUUID().toString().replace("-", "");
865     }
866
867     /**
868      * 创建租户（内部接口等，不强制短信校验）
869      */
870     @Transactional
871     public Tenant createTenant(Map<String, Object> tenantData) {
872         return doCreateTenant(tenantData, false);
873     }
874
875     /**
876      * 用户注册创建租户（公开注册，启用 dysms 时强制短信校验）
877      */
878     @Transactional
879     public Tenant registerTenant(Map<String, Object> tenantData) {
880         return doCreateTenant(tenantData, true);
881     }
882
883     private Tenant doCreateTenant(Map<String, Object> tenantData, boolean requireSmsVerification) {
884         String tenantId = generateTenantId();
885         String tenantName = (String) tenantData.get("tenantName");
886         String tenantCode = (String) tenantData.get("tenantCode");
887         String phone = (String) tenantData.get("phone");
888         String email = (String) tenantData.get("email");
889         String password = (String) tenantData.get("password");
890         String username = (String) tenantData.get("username");
891         String referralCode = null;
892         if (tenantData.get("referralCode") != null) {
893             referralCode = String.valueOf(tenantData.get("referralCode")).trim();
894         } else if (tenantData.get("inviteCode") != null) {
895             referralCode = String.valueOf(tenantData.get("inviteCode")).trim();
896         }
897         if (referralCode != null && referralCode.isEmpty()) {
898             referralCode = null;
899         }
900         Object merchantRaw = tenantData.get("merchantType");

```

```
901     if (merchantRaw == null) {
902         merchantRaw = tenantData.get("industryType");
903     }
904     String merchantType = resolveMerchantType(merchantRaw);
905
906     if (requireSmsVerification) {
907         String smsToken = (String) tenantData.get("smsToken");
908         String smsCode = (String) tenantData.get("smsCode");
909         aliyunDypnsSmsService.verifyForRegister(phone, smsToken, smsCode);
910     }
911
912     LocalDateTime now = LocalDateTime.now();
913     Tenant tenant = new Tenant();
914     tenant.setTenantId(tenantId);
915     tenant.setTenantName(tenantName);
916     tenant.setTenantCode(tenantCode);
917     tenant.setSubscriptionType(SubscriptionTierCodes.TRIAL);
918     tenant.setPhone(phone);
919     tenant.setEmail(email);
920     tenant.setEnergyBalance(initialBasicEnergy);
921     tenant.setSubStartTime(now);
922     tenant.setSubEndTime(now.plusDays(30));
923     tenant.setTenantStatus("NORMAL");
924     tenant.setMerchantType(merchantType);
925     tenant.setAccessMode("FULL");
926     tenant.setCreateTime(now);
927     tenant.setUpdateTime(now);
928
929     Tenant savedTenant = tenantRepository.save(tenant);
930
931     if (referralCode != null && !referralCode.isEmpty()) {
932         referralBindingService.bindIfPresent(referralCode, savedTenant.getTenantId(), phone);
933     }
934
935     User admin = null;
936     if (username != null && password != null) {
937         admin = userService.createUser(
938             tenantId,
939             username,
940             password,
941             "Admin",
942             email,
943             phone,
944             "ADMIN");
945     }
946
947     subscriptionLifecycleService.startTrial(savedTenant, now);
948
949     if (admin != null) {
950         referralCodeAllocator.assignIfAbsent(admin);
```

```

951     }
952
953     return tenantRepository.findById(tenantId).orElse(savedTenant);
954 }
955
956 /**
957  * 根据ID获取租户
958  */
959 public Tenant getTenantById(String tenantId) {
960     Optional<Tenant> tenant = tenantRepository.findById(tenantId);
961     return tenant.orElse(null);
962 }
963
964 /**
965  * 更新租户信息
966  */
967 @Transactional
968 public Tenant updateTenant(String tenantId, Map<String, Object> updateData) {
969     Tenant tenant = getTenantById(tenantId);
970     if (tenant == null) {
971         return null;
972     }
973
974     if (updateData.containsKey("tenantName")) {
975         tenant.setTenantName((String) updateData.get("tenantName"));
976     }
977     if (updateData.containsKey("subscriptionType")) {
978         tenant.setSubscriptionType((String) updateData.get("subscriptionType"));
979     }
980     if (updateData.containsKey("tenantStatus")) {
981         tenant.setTenantStatus((String) updateData.get("tenantStatus"));
982     }
983     if (updateData.containsKey("merchantType")) {
984         tenant.setMerchantType(resolveMerchantType(updateData.get("merchantType")));
985     }
986
987     tenant.setUpdateTime(LocalDateTime.now());
988     return tenantRepository.save(tenant);
989 }
990
991 /**
992  * 删除租户
993  */
994 private String resolveMerchantType(Object raw) {
995     if (raw == null) {
996         return MerchantTypeCodes.DEFAULT;
997     }
998     String s = String.valueOf(raw).trim();
999     if (s.isEmpty()) {
1000         return MerchantTypeCodes.DEFAULT;

```

```

1001     }
1002     String v = s.toUpperCase();
1003     if (!MerchantTypeCodes.allowed().contains(v)) {
1004         throw new IllegalArgumentException("无效的商户类型，须为字典 D013 编码之一：" + MerchantTypeCodes.allowed());
1005     }
1006     return v;
1007 }
1008
1009 @Transactional
1010 public boolean deleteTenant(String tenantId) {
1011     try {
1012         tenantRepository.deleteById(tenantId);
1013         return true;
1014     } catch (Exception e) {
1015         System.err.println("Error deleting tenant: " + e.getMessage());
1016         return false;
1017     }
1018 }
1019 }
1020
1021 /* ===== 源文件: TenantService/src/main/java/com/trademind/TenantService/controller/TenantController.java (...)
1022 package com.trademind.TenantService.controller;
1023
1024 import com.trademind.TenantService.constants.MerchantTypeCodes;
1025 import com.trademind.TenantService.model.Tenant;
1026 import com.trademind.TenantService.model.User;
1027 import com.trademind.TenantService.service.AccessModeEvaluator;
1028 import com.trademind.TenantService.service.ReferralCodeAllocator;
1029 import com.trademind.TenantService.service.TenantService;
1030 import com.trademind.TenantService.service.UserService;
1031 import com.trademind.TenantService.service.JwtUtils;
1032 import com.trademind.TenantService.service.AliyunDypnsSmsService;
1033 import com.trademind.TenantService.service.ConfigService;
1034 import org.springframework.beans.factory.annotation.Autowired;
1035 import org.springframework.beans.factory.annotation.Value;
1036 import org.springframework.http.HttpStatus;
1037 import org.springframework.http.ResponseEntity;
1038 import org.springframework.web.bind.annotation.*;
1039 import jakarta.servlet.http.HttpServletRequest;
1040
1041 import java.time.ZoneId;
1042 import java.util.HashMap;
1043 import java.util.Map;
1044
1045 /**
1046  * 租户控制器
1047  * 处理租户相关的HTTP请求
1048  */
1049 @RestController
1050 @RequestMapping("/")

```

```

1051 public class TenantController {
1052
1053     @Autowired
1054     private TenantService tenantService;
1055
1056     @Autowired
1057     private UserService userService;
1058
1059     @Autowired
1060     private ConfigService configService;
1061
1062     @Autowired
1063     private AliyunDypnsSmsService aliyunDypnsSmsService;
1064
1065     @Autowired
1066     private AccessModeEvaluator accessModeEvaluator;
1067
1068     @Autowired
1069     private ReferralCodeAllocator referralCodeAllocator;
1070
1071     @Value("${custom.security.internal-token}")
1072     private String internalToken;
1073
1074     /**
1075      * 健康检查
1076      */
1077     @GetMapping("/health")
1078     public ResponseEntity<> healthCheck() {
1079         Map<String, Object> response = new HashMap<>();
1080         response.put("status", "healthy");
1081         response.put("service", "TenantService");
1082         response.put("timestamp", System.currentTimeMillis());
1083         return ResponseEntity.ok(response);
1084     }
1085
1086     /**
1087      * 获取租户信息
1088      */
1089     @GetMapping("/info/{tenantId}")
1090     public ResponseEntity<> getTenantInfo(@PathVariable String tenantId) {
1091         try {
1092             Tenant tenant = tenantService.getTenantById(tenantId);
1093             if (tenant == null) {
1094                 return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Tenant not found");
1095             }
1096             return ResponseEntity.ok(tenant);
1097         } catch (Exception e) {
1098             System.err.println("Error getting tenant info: " + e.getMessage());
1099             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Internal server error");
1100         }

```

```

1101     }
1102
1103     /**
1104      * 创建租户
1105      */
1106     @PostMapping("/create")
1107     public ResponseEntity<?> createTenant(@RequestBody Map<String, Object> tenantData) {
1108         try {
1109             Tenant tenant = tenantService.createTenant(tenantData);
1110             return ResponseEntity.status(HttpStatus.CREATED).body(tenant);
1111         } catch (Exception e) {
1112             System.err.println("Error creating tenant: " + e.getMessage());
1113             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Internal server error");
1114         }
1115     }
1116
1117     /**
1118      * 租户注册
1119      */
1120     @PostMapping("/register")
1121     public ResponseEntity<?> registerTenant(@RequestBody Map<String, Object> registerData) {
1122         try {
1123             Tenant tenant = tenantService.registerTenant(registerData);
1124             Map<String, Object> response = new HashMap<>();
1125             response.put("success", true);
1126             response.put("message", "注册成功");
1127             response.put("tenantId", tenant.getId());
1128             response.put("merchantType", tenant.getMerchantType());
1129             Object uname = registerData.get("username");
1130             if (uname != null) {
1131                 User u = userService.getUserByUserName(String.valueOf(uname));
1132                 if (u != null) {
1133                     response.put("referralCode", u.getReferralCode());
1134                 }
1135             }
1136             return ResponseEntity.status(HttpStatus.CREATED).body(response);
1137         } catch (IllegalArgumentException e) {
1138             Map<String, Object> response = new HashMap<>();
1139             response.put("success", false);
1140             response.put("message", e.getMessage());
1141             return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
1142         } catch (Exception e) {
1143             System.err.println("Error registering tenant: " + e.getMessage());
1144             Map<String, Object> response = new HashMap<>();
1145             response.put("success", false);
1146
1147             // 处理唯一约束违反的错误
1148             String errorMessage = e.getMessage();
1149             if (errorMessage != null) {
1150                 if (errorMessage.contains("tenants_phone_key")) {

```



```

1151         response.put("message", "注册失败: 该手机号已被注册");
1152     } else if (errorMessage.contains("tenants_email_key")) {
1153         response.put("message", "注册失败: 该邮箱已被注册");
1154     } else if (errorMessage.contains("tenants_tenant_code_key")) {
1155         response.put("message", "注册失败: 该社会信用代码已被注册");
1156     } else if (errorMessage.contains("users_phone_key")) {
1157         response.put("message", "注册失败: 该手机号已被使用");
1158     } else if (errorMessage.contains("users_user_name_key")) {
1159         response.put("message", "注册失败: 该用户名已被使用");
1160     } else {
1161         response.put("message", "注册失败: " + errorMessage);
1162     }
1163 } else {
1164     response.put("message", "注册失败: 未知错误");
1165 }
1166
1167 // 对于业务逻辑错误, 返回400 Bad Request而不是500 Internal Server Error
1168 return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
1169 }
1170 }
1171
1172 /**
1173  * 用户登录
1174  */
1175 @PostMapping("/login")
1176 public ResponseEntity<?> login(@RequestBody Map<String, Object> loginData, HttpServletRequest request) {
1177     System.out.println("=== 收到登录请求 ===");
1178     System.out.println("登录请求数据: " + loginData);
1179
1180     try {
1181         String userName = (String) loginData.get("userName");
1182         String password = (String) loginData.get("password");
1183
1184         System.out.println("用户名: " + userName);
1185         System.out.println("密码: " + (password != null ? "*****" : "null"));
1186
1187         User user = userService.getUserByUserName(userName);
1188         System.out.println("查询到的用户: " + user);
1189
1190         if (user == null) {
1191             Map<String, Object> response = new HashMap<>();
1192             response.put("success", false);
1193             response.put("message", "用户名或密码错误");
1194             System.out.println("登录失败: 用户不存在");
1195             return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(response);
1196         }
1197
1198         if (!userService.verifyPassword(password, user.getPasswordHash())) {
1199             Map<String, Object> response = new HashMap<>();
1200             response.put("success", false);

```

```

1201         response.put("message", "用户名或密码错误");
1202         System.out.println("登录失败: 密码错误");
1203         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(response);
1204     }
1205
1206     String clientIp = resolveClientIp(request);
1207     userService.updateLastLoginIp(user.getUserId(), clientIp);
1208
1209     referralCodeAllocator.assignIfAbsent(user);
1210     User refreshedUser = userService.getUserByUserName(userName);
1211     if (refreshedUser != null) {
1212         user = refreshedUser;
1213     }
1214
1215     Tenant tenant = tenantService.getTenantById(user.getTenantId());
1216     String merchantType = MerchantTypeCodes.DEFAULT;
1217     if (tenant != null && tenant.getMerchantType() != null && !tenant.getMerchantType().isBlank()) {
1218         merchantType = tenant.getMerchantType();
1219     }
1220
1221     String accessMode = tenant != null ? accessModeEvaluator.evaluateAndPersist(tenant) : "FULL";
1222     long subEndMs = (tenant != null && tenant.getSubEndTime() != null)
1223         ? tenant.getSubEndTime().atZone(ZoneId.systemDefault()).toInstant().toEpochMilli()
1224         : 0L;
1225     String subscriptionTier = tenant != null && tenant.getSubscriptionType() != null
1226         ? tenant.getSubscriptionType()
1227         : "TRIAL";
1228
1229     String token = JwtUtils.generateToken(
1230         user.getUserId(),
1231         user.getUserName(),
1232         user.getTenantId(),
1233         user.getRoleType(),
1234         merchantType,
1235         accessMode,
1236         subscriptionTier,
1237         subEndMs);
1238
1239     Map<String, Object> response = new HashMap<>();
1240     response.put("success", true);
1241     response.put("message", "登录成功");
1242     response.put("token", token);
1243     response.put("expiresIn", 24 * 60 * 60); // 24小时, 单位秒
1244     response.put("user", user);
1245     response.put("merchantType", merchantType);
1246     response.put("accessMode", accessMode);
1247     response.put("referralCode", user.getReferralCode());
1248     System.out.println("登录成功: " + user.getUserName());
1249     return ResponseEntity.ok(response);
1250 } catch (Exception e) {

```

```

1251     System.err.println("Error logging in: " + e.getMessage());
1252     e.printStackTrace();
1253
1254     // 处理数据库连接错误
1255     if (e.getMessage() != null && e.getMessage().contains("Unable to acquire JDBC Connection")) {
1256         // 尝试获取当前数据库配置信息
1257         try {
1258             Map<String, Object> rdsConfig = configService.getRdsConfig();
1259             if (rdsConfig != null) {
1260                 String url = (String) rdsConfig.get("url");
1261                 String username = (String) rdsConfig.get("username");
1262                 // 掩码密码
1263                 String maskedPassword = "*****";
1264
1265                 System.err.println("【数据库连接失败】尝试连接的目标:");
1266                 System.err.println("URL: " + url);
1267                 System.err.println("Username: " + username);
1268                 System.err.println("Password: " + maskedPassword);
1269             }
1270         } catch (Exception configEx) {
1271             System.err.println("获取数据库配置信息失败: " + configEx.getMessage());
1272         }
1273     }
1274     /* ===== 源文件: TenantService/src/main/java/com/trademind/TenantService/service/UserService.java =====...
1275
1276     package com.trademind.TenantService.service;
1277
1278     import com.trademind.TenantService.model.User;
1279     import com.trademind.TenantService.repository.UserRepository;
1280     import org.springframework.beans.factory.annotation.Autowired;
1281     import org.springframework.stereotype.Service;
1282     import org.springframework.transaction.annotation.Transactional;
1283
1284     import java.security.MessageDigest;
1285     import java.security.NoSuchAlgorithmException;
1286     import java.time.LocalDateTime;
1287     import java.util.Optional;
1288
1289     /**
1290     * 用户服务类
1291     * 处理用户相关的核心业务逻辑
1292     */
1293     @Service
1294     public class UserService {
1295
1296         @Autowired
1297         private UserRepository userRepository;
1298
1299         /**
1300         * 密码加密 - 使用MD5
1301         */
1302         private String hashPassword(String password) {

```

```

1301     try {
1302         MessageDigest digest = MessageDigest.getInstance("MD5");
1303         byte[] hash = digest.digest(password.getBytes());
1304         StringBuilder hexString = new StringBuilder();
1305         for (byte b : hash) {
1306             hexString.append(String.format("%02x", b));
1307         }
1308         return hexString.toString();
1309     } catch (NoSuchAlgorithmException e) {
1310         System.err.println("Error hashing password: " + e.getMessage());
1311         throw new RuntimeException("Password hashing failed");
1312     }
1313 }
1314
1315 /**
1316  * 创建用户
1317  */
1318 @Transactional
1319 public User createUser(String tenantId, String userName, String password, String realName, String email, ...
1320     // 前端已经对密码进行了MD5加密, 直接使用
1321     String passwordHash = password;
1322
1323     User user = new User();
1324     user.setTenantId(tenantId);
1325     user.setUserName(userName);
1326     user.setPasswordHash(passwordHash);
1327     user.setRealName(realName);
1328     user.setEmail(email);
1329     user.setPhone(phone);
1330     user.setRoleType(roleType);
1331     user.setUserStatus("NORMAL");
1332     user.setCreateTime(LocalDateTime.now());
1333     user.setUpdateTime(LocalDateTime.now());
1334
1335     return userRepository.save(user);
1336 }
1337
1338 /**
1339  * 根据用户名获取用户
1340  */
1341 public User getUserByUserName(String userName) {
1342     return userRepository.findByUserName(userName);
1343 }
1344
1345 /**
1346  * 根据ID获取用户
1347  */
1348 public User getUserById(Integer userId) {
1349     Optional<User> user = userRepository.findById(userId);
1350     return user.orElse(null);

```

```

1351     }
1352
1353     /**
1354      * 验证密码 - 直接比较MD5加密后的密码
1355      */
1356     public boolean verifyPassword(String password, String passwordHash) {
1357         // 前端已经对密码进行了MD5加密, 直接比较
1358         return password.equals(passwordHash);
1359     }
1360
1361     /**
1362      * 更新用户状态
1363      */
1364     @Transactional
1365     public User updateUserStatus(Integer userId, String status) {
1366         User user = getUserById(userId);
1367         if (user == null) {
1368             return null;
1369         }
1370         user.setUserStatus(status);
1371         user.setUpdateTime(LocalDateTime.now());
1372         return userRepository.save(user);
1373     }
1374
1375     @Transactional
1376     public void updateLastLoginIp(Integer userId, String ip) {
1377         User user = getUserById(userId);
1378         if (user == null) {
1379             return;
1380         }
1381         if (ip != null && ip.length() > 45) {
1382             ip = ip.substring(0, 45);
1383         }
1384         user.setLastLoginIp(ip);
1385         user.setUpdateTime(LocalDateTime.now());
1386         userRepository.save(user);
1387     }
1388 }
1389
1390 /* ===== 源文件: TenantService/src/main/java/com/trademind/TenantService/service/AccessModeEvaluator.java =...
1391 package com.trademind.TenantService.service;
1392
1393 import com.trademind.TenantService.constants.AccessModes;
1394 import com.trademind.TenantService.model.Tenant;
1395 import com.trademind.TenantService.repository.TenantRepository;
1396 import org.springframework.beans.factory.annotation.Autowired;
1397 import org.springframework.beans.factory.annotation.Value;
1398 import org.springframework.stereotype.Service;
1399 import org.springframework.transaction.annotation.Transactional;
1400

```

```
1401 import java.time.LocalDateTime;
1402
1403 /**
1404  * 根据订阅截止时间与宽限期计算访问模式并写回租户。
1405  */
1406 @Service
1407 public class AccessModeEvaluator {
1408
1409     @Value("${custom.subscription.grace-days-after-expiry:7}")
1410     private int graceDaysAfterExpiry;
1411
1412     @Autowired
1413     private TenantRepository tenantRepository;
1414
1415     public static final String SYSTEM_OPS_TENANT_ID = "SYSTEM_OPS";
1416
1417     public String compute(Tenant tenant) {
1418         if (tenant == null) {
1419             return AccessModes.FULL;
1420         }
1421         if (SYSTEM_OPS_TENANT_ID.equals(tenant.getTenantId())) {
1422             return AccessModes.FULL;
1423         }
1424         if ("TERMINATED".equalsIgnoreCase(tenant.getTenantStatus())) {
1425             return AccessModes.BILLING_ONLY;
1426         }
1427         LocalDateTime now = LocalDateTime.now();
1428         LocalDateTime subEnd = tenant.getSubEndTime();
1429         if (subEnd != null && !now.isAfter(subEnd)) {
1430             return AccessModes.FULL;
1431         }
1432         LocalDateTime graceEnd = tenant.getGraceUntil();
1433         if (graceEnd == null && subEnd != null) {
1434             graceEnd = subEnd.plusDays(Math.max(0, graceDaysAfterExpiry));
1435         }
1436         if (graceEnd != null && !now.isAfter(graceEnd)) {
1437             return AccessModes.READ_ONLY;
1438         }
1439         return AccessModes.BILLING_ONLY;
1440     }
1441
1442     @Transactional
1443     public String evaluateAndPersist(Tenant tenant) {
1444         String mode = compute(tenant);
1445         tenant.setAccessMode(mode);
1446         tenantRepository.save(tenant);
1447         return mode;
1448     }
1449 }
1450
```

```
1451 /* ===== 源文件: TenantService/src/main/java/com/trademind/TenantService/service/SubscriptionLifecycleServi...
1452 package com.trademind.TenantService.service;
1453
1454 import com.trademind.TenantService.constants.SubscriptionTierCodes;
1455 import com.trademind.TenantService.model.SubscriptionPlan;
1456 import com.trademind.TenantService.model.Tenant;
1457 import com.trademind.TenantService.model.TenantSubscription;
1458 import com.trademind.TenantService.repository.SubscriptionPlanRepository;
1459 import com.trademind.TenantService.repository.TenantRepository;
1460 import com.trademind.TenantService.repository.TenantSubscriptionRepository;
1461 import org.springframework.beans.factory.annotation.Autowired;
1462 import org.springframework.stereotype.Service;
1463 import org.springframework.transaction.annotation.Transactional;
1464
1465 import java.math.BigDecimal;
1466 import java.time.LocalDateTime;
1467 import java.util.Optional;
1468
1469 /**
1470  * 试用开通、付费开通/续费、升级（补差价）。
1471  */
1472 @Service
1473 public class SubscriptionLifecycleService {
1474
1475     @Autowired
1476     private SubscriptionPlanRepository subscriptionPlanRepository;
1477
1478     @Autowired
1479     private TenantSubscriptionRepository tenantSubscriptionRepository;
1480
1481     @Autowired
1482     private TenantRepository tenantRepository;
1483
1484     @Autowired
1485     private ReferralQualificationService referralQualificationService;
1486
1487     @Autowired
1488     private AccessModeEvaluator accessModeEvaluator;
1489
1490     @Transactional
1491     public void startTrial(Tenant tenant, LocalDateTime now) {
1492         SubscriptionPlan trialPlan = subscriptionPlanRepository
1493             .findByMerchantTypeAndTierCode(tenant.getMerchantType(), SubscriptionTierCodes.TRIAL)
1494             .orElseThrow(() -> new IllegalStateException("缺少试用方案，请先初始化 subscription_plans"));
1495         int days = trialPlan.getTrialDays() != null ? trialPlan.getTrialDays() : 30;
1496         LocalDateTime end = now.plusDays(days);
1497         tenant.setSubscriptionType(SubscriptionTierCodes.TRIAL);
1498         tenant.setSubStartTime(now);
1499         tenant.setSubEndTime(end);
1500         tenant.setCurrentPlanId(trialPlan.getPlanId());
```

```

1501         tenantRepository.save(tenant);
1502
1503         TenantSubscription row = new TenantSubscription();
1504         row.setTenantId(tenant.getTenantId());
1505         row.setPlanId(trialPlan.getPlanId());
1506         row.setPricePaid(BigDecimal.ZERO);
1507         row.setStartedAt(now);
1508         row.setEndedAt(end);
1509         row.setStatus(TenantSubscription.STATUS_ACTIVE);
1510         tenantSubscriptionRepository.save(row);
1511         accessModeEvaluator.evaluateAndPersist(tenant);
1512     }
1513
1514     /**
1515      * 内部/支付回调：开通或续费付费档位；pricePaid>0 时触发推荐达标。
1516      */
1517     @Transactional
1518     public Tenant applyPaidPlan(String tenantId, String tierCode, int months, BigDecimal pricePaid, String ex...
1519         Tenant tenant = tenantRepository.findById(tenantId)
1520             .orElseThrow(() -> new IllegalArgumentException("租户不存在"));
1521         SubscriptionPlan plan = subscriptionPlanRepository
1522             .findByMerchantTypeAndTierCode(tenant.getMerchantType(), tierCode)
1523             .orElseThrow(() -> new IllegalArgumentException("未找到方案: " + tenant.getMerchantType() + " / " ...
1524
1525         LocalDateTime now = LocalDateTime.now();
1526         LocalDateTime baseEnd = tenant.getSubEndTime();
1527         if (baseEnd == null || baseEnd.isBefore(now)) {
1528             baseEnd = now;
1529         }
1530         LocalDateTime newEnd;
1531         if (plan.getBillingPeriodDays() != null && plan.getBillingPeriodDays() > 0) {
1532             newEnd = baseEnd.plusDays(plan.getBillingPeriodDays());
1533         } else {
1534             newEnd = baseEnd.plusMonths(Math.max(1, months));
1535         }
1536
1537         tenant.setSubscriptionType(tierCode);
1538         tenant.setCurrentPlanId(plan.getPlanId());
1539         tenant.setSubEndTime(newEnd);
1540         tenant.setTenantStatus("NORMAL");
1541         tenantRepository.save(tenant);
1542
1543         TenantSubscription row = new TenantSubscription();
1544         row.setTenantId(tenantId);
1545         row.setPlanId(plan.getPlanId());
1546         row.setPricePaid(pricePaid != null ? pricePaid : BigDecimal.ZERO);
1547         row.setStartedAt(now);
1548         row.setEndedAt(newEnd);
1549         row.setStatus(TenantSubscription.STATUS_ACTIVE);
1550         row.setExternalOrderId(externalOrderId);

```



```

1551         tenantSubscriptionRepository.save(row);
1552
1553         if (pricePaid != null && pricePaid.compareTo(BigDecimal.ZERO) > 0) {
1554             referralQualificationService.qualifyReferralIfPending(tenantId, pricePaid);
1555         }
1556         accessModeEvaluator.evaluateAndPersist(tenant);
1557         return tenant;
1558     }
1559
1560     /**
1561      * 当前付费档位续费：延长 {@link SubscriptionPlan#getBillingPeriodDays()}，实付金额默认等于方案标价。
1562      */
1563     @Transactional
1564     public Tenant renewCurrentPlan(String tenantId, BigDecimal pricePaidClient, String externalOrderId) {
1565         Tenant tenant = tenantRepository.findById(tenantId)
1566             .orElseThrow(() -> new IllegalArgumentException("租户不存在"));
1567         if (tenant.getCurrentPlanId() == null) {
1568             throw new IllegalArgumentException("当前无生效方案");
1569         }
1570         SubscriptionPlan plan = subscriptionPlanRepository.findById(tenant.getCurrentPlanId())
1571             .orElseThrow(() -> new IllegalArgumentException("方案不存在"));
1572         if (SubscriptionTierCodes.TRIAL.equals(plan.getTierCode())) {
1573             throw new IllegalArgumentException("试用版请先升级付费套餐");
1574         }
1575         BigDecimal expected = plan.getListPriceCny() != null ? plan.getListPriceCny() : BigDecimal.ZERO;
1576         BigDecimal paid = pricePaidClient != null ? pricePaidClient : expected;
1577         if (paid.add(new BigDecimal("0.009")).compareTo(expected) < 0) {
1578             throw new IllegalArgumentException("续费金额不正确，应付：" + expected.stripTrailingZeros().toPlainString());
1579         }
1580
1581         LocalDateTime now = LocalDateTime.now();
1582         LocalDateTime baseEnd = tenant.getSubEndTime();
1583         if (baseEnd == null || baseEnd.isBefore(now)) {
1584             baseEnd = now;
1585         }
1586         int days = plan.getBillingPeriodDays() != null && plan.getBillingPeriodDays() > 0
1587             ? plan.getBillingPeriodDays()
1588             : 365;
1589         LocalDateTime newEnd = baseEnd.plusDays(days);
1590
1591         tenant.setSubEndTime(newEnd);
1592         tenant.setTenantStatus("NORMAL");
1593         tenantRepository.save(tenant);
1594
1595         TenantSubscription row = new TenantSubscription();
1596         row.setTenantId(tenantId);
1597         row.setPlanId(plan.getPlanId());
1598         row.setPricePaid(paid);
1599         row.setStartedAt(now);
1600         row.setEndedAt(newEnd);

```

```

1601     row.setStatus(TenantSubscription.STATUS_ACTIVE);
1602     row.setExternalOrderId(externalOrderId);
1603     tenantSubscriptionRepository.save(row);
1604
1605     if (paid.compareTo(BigDecimal.ZERO) > 0) {
1606         referralQualificationService.qualifyReferralIfPending(tenantId, paid);
1607     }
1608     accessModeEvaluator.evaluateAndPersist(tenant);
1609     return tenant;
1610 }
1611
1612 /**
1613  * 升级到更高档位：试用/低档位按「标价差额」收款；付费档之间升级保持原到期日仅补差价。
1614  */
1615 @Transactional
1616 public Tenant upgradeToTier(String tenantId, String targetTierCode, BigDecimal pricePaidClient, String ex...
1617     Tenant tenant = tenantRepository.findById(tenantId)
1618         .orElseThrow(() -> new IllegalArgumentException("租户不存在"));
1619     if (SubscriptionTierCodes.TRIAL.equals(targetTierCode)) {
1620         throw new IllegalArgumentException("请选择付费套餐");
1621     }
1622     SubscriptionPlan target = subscriptionPlanRepository
1623         .findByMerchantTypeAndTierCode(tenant.getMerchantType(), targetTierCode)
1624         .orElseThrow(() -> new IllegalArgumentException("目标方案不存在"));
1625     if (target.getStatus() != null && target.getStatus() == 0) {
1626         throw new IllegalArgumentException("目标方案已停用");
1627     }
1628
1629     String currentTier = tenant.getSubscriptionType() != null ? tenant.getSubscriptionType() : Subscripti...
1630     int cr = tierRank(currentTier);
1631     int tr = tierRank(targetTierCode);
1632     if (tr <= cr) {
1633         throw new IllegalArgumentException("请选择高于当前等级的会员类型");
1634     }
1635
1636     SubscriptionPlan currentPlan = tenant.getCurrentPlanId() != null
1637         ? subscriptionPlanRepository.findById(tenant.getCurrentPlanId()).orElse(null)
1638         : null;
1639
1640     BigDecimal currentPrice = BigDecimal.ZERO;
1641     if (currentPlan != null && currentPlan.getListPriceCny() != null
1642         && !SubscriptionTierCodes.TRIAL.equals(currentPlan.getTierCode())) {
1643         currentPrice = currentPlan.getListPriceCny();
1644     }
1645     BigDecimal targetPrice = target.getListPriceCny() != null ? target.getListPriceCny() : BigDecimal.ZERO;
1646     BigDecimal delta = targetPrice.subtract(currentPrice);
1647     if (delta.compareTo(BigDecimal.ZERO) < 0) {
1648         delta = BigDecimal.ZERO;
1649     }
1650     BigDecimal paid = pricePaidClient != null ? pricePaidClient : delta;

```

```

1651     if (paid.add(new BigDecimal("0.009")).compareTo(delta) < 0) {
1652         throw new IllegalArgumentException("升级应付差额: " + delta.stripTrailingZeros().toPlainString());
1653     }
1654
1655     LocalDateTime now = LocalDateTime.now();
1656     LocalDateTime newEnd;
1657     boolean activePaid = currentPlan != null
1658         && !SubscriptionTierCodes.TRIAL.equals(currentTier)
1659         && tenant.getSubEndTime() != null
1660         && tenant.getSubEndTime().isAfter(now);
1661
1662     if (activePaid) {
1663         newEnd = tenant.getSubEndTime();
1664     } else {
1665         LocalDateTime base = tenant.getSubEndTime() != null && tenant.getSubEndTime().isAfter(now)
1666             ? tenant.getSubEndTime()
1667             : now;
1668         int days = target.getBillingPeriodDays() != null && target.getBillingPeriodDays() > 0
1669             ? target.getBillingPeriodDays()
1670             : 365;
1671         newEnd = base.plusDays(days);
1672     }
1673
1674     tenant.setSubscriptionType(targetTierCode);
1675     tenant.setCurrentPlanId(target.getPlanId());
1676     tenant.setSubEndTime(newEnd);
1677     tenant.setTenantStatus("NORMAL");
1678     tenantRepository.save(tenant);
1679
1680     TenantSubscription row = new TenantSubscription();
1681     row.setTenantId(tenantId);
1682     row.setPlanId(target.getPlanId());
1683     row.setPricePaid(paid);
1684     row.setStartedAt(now);
1685     row.setEndedAt(newEnd);
1686     row.setStatus(TenantSubscription.STATUS_ACTIVE);
1687     row.setExternalOrderId(externalOrderId);
1688     tenantSubscriptionRepository.save(row);
1689
1690     if (paid.compareTo(BigDecimal.ZERO) > 0) {
1691         referralQualificationService.qualifyReferralIfPending(tenantId, paid);
1692     }
1693     accessModeEvaluator.evaluateAndPersist(tenant);
1694     return tenant;
1695 }
1696
1697 private static int tierRank(String tierCode) {
1698     if (tierCode == null) {
1699         return -1;
1700     }

```

```
1701         return switch (tierCode) {
1702             case SubscriptionTierCodes.TRIAL -> 0;
1703             case SubscriptionTierCodes.BASIC -> 1;
1704             case SubscriptionTierCodes.PREMIUM -> 2;
1705             case SubscriptionTierCodes.ENTERPRISE -> 3;
1706             default -> -1;
1707         };
1708     }
1709
1710     public Optional<SubscriptionPlan> findPlan(String merchantType, String tierCode) {
1711         return subscriptionPlanRepository.findByMerchantTypeAndTierCode(merchantType, tierCode);
1712     }
1713 }
1714
1715 /* ===== 源文件: TenantService/src/main/java/com/trademind/TenantService/service/SubscriptionPaymentService...
1716 package com.trademind.TenantService.service;
1717
1718 import com.fasterxml.jackson.databind.JsonNode;
1719 import com.fasterxml.jackson.databind.ObjectMapper;
1720 import com.fasterxml.jackson.databind.node.ObjectNode;
1721 import com.trademind.TenantService.constants.SubscriptionTierCodes;
1722 import com.trademind.TenantService.model.SubscriptionPaymentEvent;
1723 import com.trademind.TenantService.model.SubscriptionPaymentOrder;
1724 import com.trademind.TenantService.model.SubscriptionPlan;
1725 import com.trademind.TenantService.model.Tenant;
1726 import com.trademind.TenantService.payment.PaymentBizTypes;
1727 import com.trademind.TenantService.payment.PaymentEventTypes;
1728 import com.trademind.TenantService.payment.PaymentOrderStatuses;
1729 import com.trademind.TenantService.payment.hccb.HccbCashierConstants;
1730 import com.trademind.TenantService.payment.hccb.HccbMerchantSigner;
1731 import com.trademind.TenantService.payment.hccb.HccbPaymentProperties;
1732 import com.trademind.TenantService.payment.hccb.HccbSignStringBuilder;
1733 import com.trademind.TenantService.payment.hccb.HccbUnifiedCashierHttpClient;
1734 import com.trademind.TenantService.repository.SubscriptionPaymentEventRepository;
1735 import com.trademind.TenantService.repository.SubscriptionPaymentOrderRepository;
1736 import com.trademind.TenantService.repository.SubscriptionPlanRepository;
1737 import com.trademind.TenantService.repository.TenantRepository;
1738 import org.slf4j.Logger;
1739 import org.slf4j.LoggerFactory;
1740 import org.springframework.beans.factory.annotation.Autowired;
1741 import org.springframework.stereotype.Service;
1742 import org.springframework.transaction.annotation.Transactional;
1743
1744 import java.math.BigDecimal;
1745 import java.math.RoundingMode;
1746 import java.net.URLDecoder;
1747 import java.net.http.HttpResponse;
1748 import java.nio.charset.StandardCharsets;
1749 import java.security.SecureRandom;
1750 import java.time.LocalDateTime;
```

```
1751 import java.time.format.DateTimeFormatter;
1752 import java.util.HashMap;
1753 import java.util.LinkedHashMap;
1754 import java.util.List;
1755 import java.util.Map;
1756 import java.util.Optional;
1757 import java.util.TreeMap;
1758 import java.util.UUID;
1759
1760 @Service
1761 public class SubscriptionPaymentService {
1762
1763     private static final Logger log = LoggerFactory.getLogger(SubscriptionPaymentService.class);
1764     private static final DateTimeFormatter TXN_TIME = DateTimeFormatter.ofPattern("yyyyMMddHHmmss");
1765     private static final ObjectMapper OM = new ObjectMapper();
1766     private static final SecureRandom RND = new SecureRandom();
1767
1768     @Autowired
1769     private HccbPaymentProperties hccbPaymentProperties;
1770
1771     @Autowired
1772     private SubscriptionPaymentOrderRepository paymentOrderRepository;
1773
1774     @Autowired
1775     private SubscriptionPaymentEventRepository paymentEventRepository;
1776
1777     @Autowired
1778     private TenantRepository tenantRepository;
1779
1780     @Autowired
1781     private SubscriptionPlanRepository subscriptionPlanRepository;
1782
1783     @Autowired
1784     private SubscriptionLifecycleService subscriptionLifecycleService;
1785
1786     @Autowired
1787     private HccbMerchantSigner hccbMerchantSigner;
1788
1789     @Autowired
1790     private HccbUnifiedCashierHttpClient hccbUnifiedCashierHttpClient;
1791
1792     public boolean isPaymentEnabled() {
1793         return hccbPaymentProperties.isEnabled();
1794     }
1795
1796     public Optional<SubscriptionPaymentOrder> findByTxnOrderId(String txnOrderId) {
1797         return paymentOrderRepository.findByTxnOrderId(txnOrderId);
1798     }
1799
1800     /**
```

```

1801 * 创建订阅支付单并调用（或模拟）统一收银台下支付。
1802 *
1803 * @param action NEW / RENEW / UPGRADE
1804 */
1805 @Transactional
1806 public Map<String, Object> createPayOrder(String tenantId, int userId, String action, String targetTierCode)
1807     throws Exception {
1808     if (!hccbPaymentProperties.isEnabled()) {
1809         throw new IllegalStateException("收银台未启用");
1810     }
1811     Tenant tenant = tenantRepository.findById(tenantId)
1812         .orElseThrow(() -> new IllegalArgumentException("租户不存在"));
1813     String mt = tenant.getMerchantType() != null ? tenant.getMerchantType() : "WHOLESALE";
1814     String tier = targetTierCode != null ? targetTierCode.trim().toUpperCase() : "";
1815
1816     String bizType;
1817     long amountCents;
1818     SubscriptionPlan targetPlan;
1819
1820     String sub = tenant.getSubscriptionType() != null ? tenant.getSubscriptionType() : SubscriptionTierCo...
1821     LocalDateTime now = LocalDateTime.now();
1822
1823     switch (action == null ? "" : action.trim().toUpperCase()) {
1824         case "NEW" -> {
1825             if (!SubscriptionTierCodes.TRIAL.equals(sub)) {
1826                 throw new IllegalArgumentException("仅试用租户可发起新购支付");
1827             }
1828             if (!SubscriptionTierCodes.BASIC.equals(tier) && !SubscriptionTierCodes.PREMIUM.equals(tier)) {
1829                 throw new IllegalArgumentException("请选择启航或优享档位");
1830             }
1831             bizType = PaymentBizTypes.SUB_NEW;
1832             targetPlan = subscriptionPlanRepository.findByMerchantTypeAndTierCode(mt, tier)
1833                 .orElseThrow(() -> new IllegalArgumentException("未找到方案"));
1834             amountCents = yuanToCents(targetPlan.getListPriceCny());
1835         }
1836         case "RENEW" -> {
1837             if (SubscriptionTierCodes.TRIAL.equals(sub)) {
1838                 throw new IllegalArgumentException("试用版请使用新购");
1839             }
1840             if (tenant.getCurrentPlanId() == null) {
1841                 throw new IllegalArgumentException("当前无生效方案");
1842             }
1843             SubscriptionPlan current = subscriptionPlanRepository.findById(tenant.getCurrentPlanId())
1844                 .orElseThrow(() -> new IllegalArgumentException("方案不存在"));
1845             if (SubscriptionTierCodes.TRIAL.equals(current.getTierCode())) {
1846                 throw new IllegalArgumentException("试用版请使用新购");
1847             }
1848             bizType = PaymentBizTypes.SUB_RENEW;
1849             targetPlan = current;
1850             tier = current.getTierCode();

```

```

1851         amountCents = yuanToCents(current.getListPriceCny());
1852     }
1853     case "UPGRADE" -> {
1854         if (!SubscriptionTierCodes.BASIC.equals(sub)) {
1855             throw new IllegalArgumentException("仅启航会员可升级到优享");
1856         }
1857         if (!SubscriptionTierCodes.PREMIUM.equals(tier)) {
1858             throw new IllegalArgumentException("升级目标应为优享");
1859         }
1860         bizType = PaymentBizTypes.SUB_UPGRADE;
1861         SubscriptionPlan basic = subscriptionPlanRepository
1862             .findByMerchantTypeAndTierCode(mt, SubscriptionTierCodes.BASIC)
1863             .orElseThrow(() -> new IllegalArgumentException("缺少启航方案"));
1864         targetPlan = subscriptionPlanRepository
1865             .findByMerchantTypeAndTierCode(mt, SubscriptionTierCodes.PREMIUM)
1866             .orElseThrow(() -> new IllegalArgumentException("缺少优享方案"));
1867         BigDecimal delta = targetPlan.getListPriceCny() != null && basic.getListPriceCny() != null
1868             ? targetPlan.getListPriceCny().subtract(basic.getListPriceCny())
1869             : BigDecimal.ZERO;
1870         if (delta.compareTo(BigDecimal.ZERO) < 0) {
1871             delta = BigDecimal.ZERO;
1872         }
1873         amountCents = yuanToCents(delta);
1874     }
1875     default -> throw new IllegalArgumentException("不支持的 action: " + action);
1876 }
1877
1878 if (amountCents <= 0) {
1879     throw new IllegalArgumentException("应付金额无效");
1880 }
1881
1882 String txnOrderId = generateTxnOrderId();
1883 SubscriptionPaymentOrder order = new SubscriptionPaymentOrder();
1884 order.setTenantId(tenantId);
1885 order.setUserId(userId);
1886 order.setTxnOrderId(txnOrderId);
1887 order.setBizType(bizType);
1888 order.setTargetTierCode(tier);
1889 order.setPlanId(targetPlan.getPlanId());
1890 order.setAmountCents(amountCents);
1891 order.setCurrency("CNY");
1892 order.setStatus(PaymentOrderStatuses.PENDING);
1893 paymentOrderRepository.save(order);
1894
1895 Map<String, String> payFields = buildPayFields(order, targetPlan);
1896
1897 if (!hccbPaymentProperties.isOutboundEnabled()) {
1898     String mockUrl = "https://test.hccb.cc/ifsp-payweb/mobile/uniCashier/uniCashier.html?tokenCode=MO...
1899         + txnOrderId + "&uniCashierPayType=null";
1900     order.setStatus(PaymentOrderStatuses.PAYING);

```

```

1901     order.setChannelRespCode("0000");
1902     order.setChannelRespMsg("MOCK_OUTBOUND_DISABLED");
1903     order.setRespTxnSsn("MOCK_" + txnOrderId);
1904     order.setRespTxnTime(LocalDate.now().format(TXN_TIME));
1905     order.setHccbTokenCode("MOCK");
1906     paymentOrderRepository.save(order);
1907     // DEBUG: 请求报文摘要写入 channel_resp_msg, 联调完成后可改回 null
1908     appendEvent(order.getId(), PaymentEventTypes.TXN_ACCEPT_REQUEST, 200, null, summarize(payFields),...
1909     appendEvent(order.getId(), PaymentEventTypes.TXN_ACCEPT_RESPONSE, 200, "0000", "MOCK", mockUrl);
1910     // 未外呼行方时直接履约, 便于本地/联调 UI; 生产务必 outboundEnabled=true 并走异步通知
1911     fulfillPaidOrder(order);
1912     return Map.of(
1913         "success", true,
1914         "mock", true,
1915         "txnOrderId", txnOrderId,
1916         "payUrl", mockUrl,
1917         "amountCents", amountCents,
1918         "bizType", bizType,
1919         "instantComplete", true
1920     );
1921 }
1922
1923 if (!hccbMerchantSigner.isConfigured(hccbPaymentProperties)) {
1924     throw new IllegalStateException("已开启 outbound 但未配置商户 PKCS12 证书路径");
1925 }
1926
1927 LinkedHashMap<String, Object> body = new LinkedHashMap<>();
1928 for (Map.Entry<String, String> e : payFields.entrySet()) {
1929     body.put(e.getKey(), e.getValue());
1930 }
1931 String canonical = HccbSignStringBuilder.buildCanonical(body);
1932 String sign = hccbMerchantSigner.signCanonical(hccbPaymentProperties, canonical);
1933 body.put("signAture", sign);
1934
1935 String json = OM.writeValueAsString(body);
1936 // DEBUG: 实际 POST 报文写入 channel_resp_msg, 联调完成后可去除
1937 appendEvent(order.getId(), PaymentEventTypes.TXN_ACCEPT_REQUEST, 200, null, json, null);
1938 HttpResponse<String> httpResp = hccbUnifiedCashierHttpClient.postTxnAccept(hccbPaymentProperties, json);
1939 appendEvent(order.getId(), PaymentEventTypes.TXN_ACCEPT_RESPONSE, httpResp.statusCode(), null, null,
1940     summarize(httpResp.body()));
1941
1942 JsonNode root = OM.readTree(httpResp.body());
1943 String respCode = text(root, "respCode");
1944 String respMsg = text(root, "respMsg");
1945 order.setChannelRespCode(respCode);
1946 order.setChannelRespMsg(respMsg != null && respMsg.length() > 1000 ? respMsg.substring(0, 1000) : res...
1947
1948 if (!"0000".equals(respCode)) {
1949     order.setStatus(PaymentOrderStatuses.FAILED);
1950     paymentOrderRepository.save(order);

```



```

1951         String code = respCode != null ? respCode : "";
1952         String msg = respMsg != null ? respMsg : "";
1953         Map<String, Object> err = new HashMap<>();
1954         err.put("success", false);
1955         err.put("txnOrderId", txnOrderId);
1956         err.put("respCode", code);
1957         err.put("respMsg", msg);
1958         err.put("message", msg.isEmpty() ? ("行方返回码: " + code) : msg);
1959         return err;
1960     }
1961
1962     String tokenJson = text(root, "txnTokenPayData");
1963     String payUrl = null;
1964     String tokenCode = null;
1965     if (tokenJson != null && !tokenJson.isBlank()) {
1966         JsonNode inner = OM.readTree(tokenJson);
1967         payUrl = text(inner, "tokenCodeURL");
1968         tokenCode = text(inner, "tokenCode");
1969     }
1970     order.setStatus(PaymentOrderStatuses.PAYING);
1971     order.setRespTxnSsn(text(root, "respTxnSsn"));
1972     order.setRespTxnTime(text(root, "respTxnTime"));
1973     order.setHccbTokenCode(tokenCode);
1974     paymentOrderRepository.save(order);
1975
1976     Map<String, Object> res = new HashMap<>();
1977     res.put("success", true);
1978     res.put("mock", false);
1979     res.put("txnOrderId", txnOrderId);
1980     res.put("payUrl", payUrl != null ? payUrl : "");
1981     res.put("amountCents", amountCents);
1982     res.put("bizType", bizType);
1983     return res;
1984 }
1985
1986 private Map<String, String> buildPayFields(SubscriptionPaymentOrder order, SubscriptionPlan plan) {
1987     Map<String, String> m = new TreeMap<>();
1988     m.put("encoding", "UTF-8");
1989     m.put("txnType", "1000");
1990     m.put("txnSubType", "100010");
1991     m.put("aesWay", hccbPaymentProperties.getAesWay());
1992     m.put("sdkAppId", hccbPaymentProperties.getSdkAppId());
1993     m.put("merId", hccbPaymentProperties.getMerId());
1994     m.put("certId", hccbPaymentProperties.getCertId());
1995     m.put("backEndUrl", hccbPaymentProperties.getNotifyUrl());
1996
1997     /* ===== 源文件: RDSservice/src/main/java/com/trademind/RDSservice/service/OrderService.java ===== */
1998     package com.trademind.RDSservice.service;
1999
2000     import com.trademind.RDSservice.model.Order;

```

```

2001 import com.trademind.RDService.model.OrderItem;
2002 import com.trademind.RDService.model.OrderWithItemsDTO;
2003 import com.trademind.RDService.model.Product;
2004 import com.trademind.RDService.ledger.AccountLedgerAppender;
2005 import com.trademind.RDService.repository.OrderItemRepository;
2006 import com.trademind.RDService.repository.OrderRepository;
2007 import com.trademind.RDService.repository.ProductRepository;
2008 import com.trademind.RDService.context.UserContext;
2009 import org.slf4j.Logger;
2010 import org.slf4j.LoggerFactory;
2011 import org.springframework.beans.factory.annotation.Autowired;
2012 import org.springframework.stereotype.Service;
2013 import org.springframework.transaction.annotation.Transactional;
2014
2015 import java.math.BigDecimal;
2016 import java.sql.Timestamp;
2017 import java.time.LocalDateTime;
2018 import java.util.HashMap;
2019 import java.util.List;
2020 import java.util.Map;
2021 import java.util.UUID;
2022
2023 /**
2024  * 订单服务类
2025  * 实现订单的业务逻辑
2026  */
2027 @Service
2028 public class OrderService {
2029
2030     private static final Logger logger = LoggerFactory.getLogger(OrderService.class);
2031     private final OrderRepository orderRepository;
2032     private final OrderItemRepository orderItemRepository;
2033     private final ProductRepository productRepository;
2034     private final AccountLedgerAppender accountLedgerAppender;
2035
2036     @Autowired
2037     public OrderService(OrderRepository orderRepository, OrderItemRepository orderItemRepository,
2038         ProductRepository productRepository, AccountLedgerAppender accountLedgerAppender) {
2039         this.orderRepository = orderRepository;
2040         this.orderItemRepository = orderItemRepository;
2041         this.productRepository = productRepository;
2042         this.accountLedgerAppender = accountLedgerAppender;
2043     }
2044
2045     /**
2046     * 创建订单
2047     * 事务处理：1. 写入orders表；2. 批量写入order_items表；3. 根据product_id同步更新products表的stock（扣减库存）
2048     * @param orderWithItemsDTO 订单及详情DTO
2049     * @return 创建的订单
2050     */

```

```

2051     @Transactional
2052     public Order createOrder(OrderWithItemsDTO orderWithItemsDTO) {
2053         logger.info("=== 开始创建订单 ===");
2054
2055         // 获取订单主表信息
2056         Order order = orderWithItemsDTO.getOrder();
2057         List<OrderItem> orderItems = orderWithItemsDTO.getOrderItems();
2058
2059         // 生成订单编号
2060         String orderCode = generateOrderCode();
2061         order.setOrderCode(orderCode);
2062
2063         // 设置创建时间和更新时间
2064         LocalDateTime now = LocalDateTime.now();
2065         order.setCreateTime(now);
2066         order.setUpdateTime(now);
2067
2068         // 保存订单主表
2069         Order savedOrder = orderRepository.save(order);
2070         logger.info("订单主表保存成功, 订单ID: {}", savedOrder.getId());
2071
2072         // 批量保存订单详情
2073         for (OrderItem orderItem : orderItems) {
2074             // 设置订单ID
2075             orderItem.setOrderId(savedOrder.getId());
2076             // 设置创建时间和更新时间
2077             orderItem.setCreateTime(now);
2078             orderItem.setUpdateTime(now);
2079
2080             // 兜底填充单价, 避免 unit_price 非空约束失败
2081             if (orderItem.getUnitPrice() == null) {
2082                 BigDecimal totalAmount = orderItem.getTotalAmount() == null ? BigDecimal.ZERO : orderItem.get...
2083                 Integer qty = orderItem.getQuantity() == null || orderItem.getQuantity() <= 0 ? 1 : orderItem...
2084                 orderItem.setUnitPrice(totalAmount.divide(BigDecimal.valueOf(qty), 2, java.math.RoundingMode...
2085             }
2086
2087             // 保存订单详情
2088             orderItemRepository.save(orderItem);
2089             logger.info("订单详情保存成功, 订单项ID: {}", orderItem.getId());
2090
2091             // 根据product_id同步更新products表的stock(扣减库存)
2092             Product product = productRepository.findByProductId(orderItem.getProductId());
2093             if (product != null) {
2094                 int currentStock = product.getStock();
2095                 int quantity = orderItem.getQuantity();
2096
2097                 // 检查库存是否足够
2098                 if (currentStock < quantity) {
2099                     throw new RuntimeException("产品ID " + product.getProductId() + " 库存不足, 当前库存: " + currentSto...
2100                 }

```

```

2101
2102         // 扣减库存
2103         product.setStock(currentStock - quantity);
2104         product.setUpdateTime(now);
2105         productRepository.save(product);
2106         logger.info("产品ID {} 库存扣减成功, 当前库存: {}", product.getId(), product.getStock());
2107     } else {
2108         throw new RuntimeException("产品ID " + orderItem.getId() + " 不存在");
2109     }
2110 }
2111
2112     logger.info("=== 订单创建完成 ===");
2113     return savedOrder;
2114 }
2115
2116 /**
2117  * 根据订单ID查询订单
2118  * @param orderId 订单ID
2119  * @return 订单
2120  */
2121 public Order getOrderById(Integer orderId) {
2122     return orderRepository.findById(orderId).orElse(null);
2123 }
2124
2125 /**
2126  * 根据订单编号查询订单
2127  * @param orderCode 订单编号
2128  * @return 订单
2129  */
2130 public Order getOrderByOrderCode(String orderCode) {
2131     return orderRepository.findByOrderCode(orderCode);
2132 }
2133
2134 /**
2135  * 根据租户ID查询订单
2136  * @param tenantId 租户ID
2137  * @return 订单列表
2138  */
2139 public List<Order> getOrdersByTenantId(String tenantId) {
2140     return orderRepository.findByTenantId(tenantId);
2141 }
2142
2143 /**
2144  * 根据租户ID查询最新的10条订单
2145  * @param tenantId 租户ID
2146  * @return 订单列表
2147  */
2148 public List<Order> getLatestOrdersByTenantId(String tenantId) {
2149     logger.info("=== 查询最新10条订单 ===");
2150     logger.info("TenantId: {}", tenantId);

```

```

2151         logger.info("SQL: SELECT * FROM orders WHERE tenant_id = ? ORDER BY create_time DESC LIMIT 10");
2152         return orderRepository.findTop10ByTenantIdOrderByCreateTimeDesc(tenantId);
2153     }
2154
2155     /**
2156     * 根据租户ID和订单状态查询订单
2157     * @param tenantId 租户ID
2158     * @param orderStatus 订单状态
2159     * @return 订单列表
2160     */
2161     public List<Order> getOrdersByTenantIdAndStatus(String tenantId, String orderStatus) {
2162         return orderRepository.findByTenantIdAndOrderStatus(tenantId, orderStatus);
2163     }
2164
2165     /**
2166     * 根据租户ID和多个订单状态查询订单（进行中订单）
2167     * @param tenantId 租户ID
2168     * @param orderStatuses 状态集合
2169     * @return 订单列表
2170     */
2171     public List<Order> getOrdersByTenantIdAndStatuses(String tenantId, List<String> orderStatuses) {
2172         return orderRepository.findByTenantIdAndOrderStatusInOrderByCreateTimeDesc(tenantId, orderStatuses);
2173     }
2174
2175     /**
2176     * 根据客户ID查询订单
2177     * @param custId 客户ID
2178     * @return 订单列表
2179     */
2180     public List<Order> getOrdersByCustId(Integer custId) {
2181         String tenantId = UserContext.getTenantId();
2182         logger.info("=== 根据客户ID查询订单 ===");
2183         logger.info("CustId: {}", custId);
2184         logger.info("TenantId: {}", tenantId);
2185         logger.info("SQL: SELECT order_code, total_amount, order_status, create_time FROM orders WHERE cust_i...");
2186         return orderRepository.findTop10ByCustIdAndTenantIdOrderByCreateTimeDesc(custId, tenantId);
2187     }
2188
2189     /**
2190     * 更新订单状态
2191     * @param orderId 订单ID
2192     * @param orderStatus 新的订单状态
2193     * @return 更新后的订单
2194     */
2195     public Order updateOrderStatus(Integer orderId, String orderStatus) {
2196         Order order = orderRepository.findById(orderId).orElse(null);
2197         if (order != null) {
2198             String oldStatus = order.getOrderStatus();
2199             order.setOrderStatus(orderStatus);
2200             order.setUpdateTime(LocalDateTime.now());

```

```

2201         Order saved = orderRepository.save(order);
2202         accountLedgerAppender.onOrderStatusChanged(saved, oldStatus, orderStatus);
2203         return saved;
2204     }
2205     return null;
2206 }
2207
2208 /**
2209  * 根据订单ID查询订单详情
2210  * @param orderId 订单ID
2211  * @return 订单详情列表
2212  */
2213 public List<OrderItem> getOrderItemsByOrderId(Integer orderId) {
2214     return orderItemRepository.findByOrderId(orderId);
2215 }
2216
2217 /**
2218  * 时间窗内各产品订单销售数量合计（基本单位 = order_items.quantity）
2219  */
2220 public Map<Integer, Long> sumSoldQuantitiesForProducts(String tenantId, LocalDateTime from, LocalDateTime...
2221     if (tenantId == null || tenantId.isBlank() || productIds == null || productIds.isEmpty()) {
2222         return new HashMap<>();
2223     }
2224     List<Integer> distinct = productIds.stream().distinct().toList();
2225     List<Object[]> rows = orderItemRepository.sumQuantitiesByProductInWindow(
2226         tenantId, Timestamp.valueOf(from), Timestamp.valueOf(to), distinct);
2227     Map<Integer, Long> map = new HashMap<>();
2228     for (Object[] row : rows) {
2229         map.put(((Number) row[0]).intValue(), ((Number) row[1]).longValue());
2230     }
2231     return map;
2232 }
2233
2234 /**
2235  * 生成订单编号
2236  * 格式: ORD + 年月日时分秒 + 6位随机数
2237  * @return 订单编号
2238  */
2239 private String generateOrderCode() {
2240     String timestamp = LocalDateTime.now().format(java.time.format.DateTimeFormatter.ofPattern("yyyyMMddH...
2241     String random = UUID.randomUUID().toString().substring(0, 6).toUpperCase();
2242     return "ORD" + timestamp + random;
2243 }
2244 }
2245
2246 /* ===== 源文件: RDSservice/src/main/java/com/trademind/RDSservice/service/ProductService.java ===== */
2247 package com.trademind.RDSservice.service;
2248
2249 import com.trademind.RDSservice.model.Product;
2250 import com.trademind.RDSservice.model.UnitConversion;

```



```

2301         }
2302     }
2303 }
2304 if (product.getUserId() == null) {
2305     throw new RuntimeException("用户ID不存在");
2306 }
2307
2308 product.setCreateTime(LocalDateTime.now());
2309 product.setUpdateTime(LocalDateTime.now());
2310
2311 Product savedProduct = productRepository.save(product);
2312
2313 // 保存单位换算记录
2314 if (product.getUnitConversions() != null && !product.getUnitConversions().isEmpty()) {
2315     for (UnitConversion conversion : product.getUnitConversions()) {
2316         conversion.setProductId(savedProduct.getProductId());
2317         conversion.setTenantId(tenantId);
2318         unitConversionRepository.save(conversion);
2319     }
2320 }
2321
2322 logger.info("产品创建成功, 产品ID: {}, 产品名称: {}", savedProduct.getProductId(), savedProduct.getName());
2323 return savedProduct;
2324 }
2325
2326 /**
2327  * 根据ID获取产品
2328  */
2329 public Product getProductById(Integer productId) {
2330     return productRepository.findById(productId).orElse(null);
2331 }
2332
2333 /**
2334  * 根据租户ID和产品ID获取产品
2335  */
2336 public Product getProductByTenantIdAndProductId(String tenantId, Integer productId) {
2337     return productRepository.findByTenantIdAndProductId(tenantId, productId);
2338 }
2339
2340 /**
2341  * 根据租户ID获取产品列表
2342  */
2343 public List<Product> getProductsByTenantId(String tenantId) {
2344     return productRepository.findByTenantId(tenantId);
2345 }
2346
2347 /**
2348  * 根据租户ID和分类获取产品列表
2349  */
2350 public List<Product> getProductsByTenantIdAndCategoryId(String tenantId, Integer categoryId) {

```



```

2351         return productRepository.findByTenantIdAndCategoryId(tenantId, categoryId);
2352     }
2353
2354     /**
2355      * 更新产品
2356      */
2357     @Transactional
2358     public Product updateProduct(Product product) {
2359         String tenantId = UserContext.getTenantId();
2360         if (tenantId == null || tenantId.isBlank()) {
2361             throw new RuntimeException("租户ID不存在");
2362         }
2363         if (!tenantId.equals(product.getTenantId())) {
2364             throw new RuntimeException("租户ID不匹配");
2365         }
2366
2367         // 检查产品是否存在
2368         Product existingProduct = productRepository.findByTenantIdAndProductId(tenantId, product.getProductId());
2369         if (existingProduct == null) {
2370             throw new RuntimeException("产品不存在");
2371         }
2372         if (product.getUserId() == null) {
2373             product.setUserId(existingProduct.getUserId());
2374         }
2375         if (product.getCreateTime() == null) {
2376             product.setCreateTime(existingProduct.getCreateTime());
2377         }
2378
2379         product.setUpdateTime(LocalDateTime.now());
2380
2381         Product updatedProduct = productRepository.save(product);
2382
2383         // 先删除原有的单位换算记录，再添加新的
2384         unitConversionRepository.deleteByTenantIdAndProductId(tenantId, product.getProductId());
2385         if (product.getUnitConversions() != null && !product.getUnitConversions().isEmpty()) {
2386             for (UnitConversion conversion : product.getUnitConversions()) {
2387                 conversion.setProductId(product.getProductId());
2388                 conversion.setTenantId(tenantId);
2389                 unitConversionRepository.save(conversion);
2390             }
2391         }
2392
2393         logger.info("产品更新成功, 产品ID: {}, 产品名称: {}", updatedProduct.getProductId(), updatedProduct.getName());
2394         return updatedProduct;
2395     }
2396
2397     /**
2398      * 删除产品（物理删除）
2399      */
2400     @Transactional

```

```

2401 public boolean deleteProduct(Integer productId) {
2402     try {
2403         String tenantId = UserContext.getTenantId();
2404         Product product = productRepository.findByTenantIdAndProductId(tenantId, productId);
2405         if (product != null) {
2406             // 物理删除产品
2407             productRepository.delete(product);
2408
2409             logger.info("产品删除成功, 产品ID: {}", productId);
2410             return true;
2411         }
2412         return false;
2413     } catch (Exception e) {
2414         logger.error("产品删除失败, 产品ID: {}, 错误信息: {}", productId, e.getMessage());
2415         return false;
2416     }
2417 }
2418
2419 /**
2420  * 根据租户ID获取库存不足的产品列表
2421  */
2422 public List<Product> getLowStockProducts(String tenantId) {
2423     return productRepository.findLowStockProducts(tenantId);
2424 }
2425
2426 /**
2427  * 根据租户ID获取销售量排名靠前的产品列表
2428  */
2429 public List<Product> getTopSellingProducts(String tenantId, Integer limit) {
2430     return productRepository.findTopSellingProducts(tenantId, limit);
2431 }
2432
2433 /**
2434  * 根据租户ID和SKU检查产品是否存在
2435  */
2436 public boolean existsByTenantIdAndSku(String tenantId, String sku) {
2437     return productRepository.findByTenantIdAndSku(tenantId, sku) != null;
2438 }
2439
2440 /**
2441  * 获取进货建议列表
2442  */
2443 public List<Map<String, Object>> getRestockSuggestions(String tenantId) {
2444     // 1. 查询所有 stock <= warning_stock 的产品并关联供应商信息
2445     List<Map<String, Object>> lowStockProducts = productRepository.findLowStockProductsWithSupplier(tenan...
2446
2447     // 2. 按供应商分组
2448     Map<Integer, List<Map<String, Object>>> supplierGroups = new HashMap<>();
2449
2450     for (Map<String, Object> product : lowStockProducts) {

```

```
2451 Integer supplierId = (Integer) product.get("supplier_id");
2452 if (supplierId == null) {
2453     continue;
2454 }
2455
2456 if (!supplierGroups.containsKey(supplierId)) {
2457     supplierGroups.put(supplierId, new ArrayList<>());
2458 }
2459
2460 // 3. 计算建议采购量 = (sales_volume * 1.5) - stock
2461 Integer salesVolume = (Integer) product.get("sales_volume");
2462 if (salesVolume == null) {
2463     salesVolume = 0;
2464 }
2465 Integer stock = (Integer) product.get("stock");
2466 if (stock == null) {
2467     stock = 0;
2468 }
2469 int suggestedQty = (int) (salesVolume * 1.5) - stock;
2470 if (suggestedQty < 0) {
2471     suggestedQty = 0;
2472 }
2473 product.put("calculated_qty", suggestedQty);
2474
2475 // 4. 计算预估金额
2476 BigDecimal price = (BigDecimal) product.get("price");
2477 if (price == null) {
2478     price = BigDecimal.ZERO;
2479 }
2480 BigDecimal estimatedAmount = price.multiply(BigDecimal.valueOf(suggestedQty));
2481 product.put("estimated_amount", estimatedAmount);
2482
2483 // 5. 添加所需的字段名称
2484 product.put("productName", product.get("name"));
2485 product.put("currentStock", product.get("stock"));
2486 product.put("warningStock", product.get("warning_stock"));
2487 product.put("supplierName", product.get("supplier_name"));
2488
2489 supplierGroups.get(supplierId).add(product);
2490 }
2491
2492 // 5. 汇总每个供应商的订单总计
2493 List<Map<String, Object>> suggestions = new ArrayList<>();
2494 for (Map.Entry<Integer, List<Map<String, Object>>> entry : supplierGroups.entrySet()) {
2495     Integer supplierId = entry.getKey();
2496     List<Map<String, Object>> products = entry.getValue();
2497
2498     if (products.isEmpty()) {
2499         continue;
2500     }
```

```

2501
2502 // 获取供应商名称
2503 String supplierName = (String) products.get(0).get("supplier_name");
2504 if (supplierName == null) {
2505     supplierName = "未知供应商";
2506 }
2507
2508 // 计算订单总计
2509 BigDecimal totalAmount = BigDecimal.ZERO;
2510 for (Map<String, Object> product : products) {
2511     BigDecimal estimatedAmount = (BigDecimal) product.get("estimated_amount");
2512     if (estimatedAmount != null) {
2513         totalAmount = totalAmount.add(estimatedAmount);
2514     }
2515 }
2516
2517 // 创建供应商进货建议
2518 Map<String, Object> supplierSuggestion = new HashMap<>();
2519 supplierSuggestion.put("supplier_id", supplierId);
2520 supplierSuggestion.put("supplier_name", supplierName);
2521 supplierSuggestion.put("products", products);
2522 supplierSuggestion.put("order_total", totalAmount);
2523
2524 suggestions.add(supplierSuggestion);
2525 }
2526
2527 return suggestions;
2528 }
2529
2530 /**
2531  * 批量更新产品库存
2532  */
2533 @Transactional
2534 public boolean batchUpdateStock(String tenantId, Map<Integer, Integer> stockChanges) {
2535     try {
2536         for (Map.Entry<Integer, Integer> entry : stockChanges.entrySet()) {
2537             Integer productId = entry.getKey();
2538             Integer change = entry.getValue();
2539
2540             Product product = productRepository.findByTenantIdAndProductId(tenantId, productId);
2541             if (product == null) {
2542                 throw new RuntimeException("产品不存在: " + productId);
2543             }
2544
2545             Integer newStock = product.getStock() + change;
2546             if (newStock < 0) {
2547                 throw new RuntimeException("库存不足, 产品ID: " + productId);
2548             }
2549
2550             product.setStock(newStock);

```

```

2551         product.setUpdateTime(LocalDateTime.now());
2552         productRepository.save(product);
2553
2554         logger.info("产品库存更新成功, 产品ID: {}, 变更数量: {}, 新库存: {}", productId, change, newStock);
2555     }
2556     return true;
2557 } catch (Exception e) {
2558     logger.error("批量更新库存失败: {}", e.getMessage());
2559     throw new RuntimeException("批量更新库存失败: " + e.getMessage());
2560 }
2561 }
2562 }
2563
2564 /* ===== 源文件: RDSservice/src/main/java/com/trademind/RDSservice/controller/OrderController.java ===== */
2565 package com.trademind.RDSservice.controller;
2566
2567 import com.trademind.RDSservice.model.Order;
2568 import com.trademind.RDSservice.model.OrderItem;
2569 import com.trademind.RDSservice.model.OrderWithItemsDTO;
2570 import com.trademind.RDSservice.service.OrderService;
2571 import com.trademind.RDSservice.context.UserContext;
2572 import org.slf4j.Logger;
2573 import org.slf4j.LoggerFactory;
2574 import org.springframework.beans.factory.annotation.Autowired;
2575 import org.springframework.web.bind.annotation.*;
2576
2577 import java.util.ArrayList;
2578 import java.util.HashMap;
2579 import java.util.List;
2580 import java.util.Map;
2581 import java.time.LocalDateTime;
2582 import java.util.Arrays;
2583
2584 /**
2585  * 订单控制器
2586  * 处理订单相关的HTTP请求
2587  */
2588 @RestController
2589 @RequestMapping("/api/v1/rd/orders")
2590 public class OrderController {
2591
2592     private static final Logger logger = LoggerFactory.getLogger(OrderController.class);
2593     private final OrderService orderService;
2594
2595     @Autowired
2596     public OrderController(OrderService orderService) {
2597         this.orderService = orderService;
2598     }
2599
2600     /**

```

```

2601 * 创建订单
2602 * 接收订单主表及详情列表, 处理订单保存和库存更新
2603 * @param orderWithItemsDTO 订单及详情DTO
2604 * @param tenantId 租户ID
2605 * @return 创建结果
2606 */
2607 @PostMapping
2608 public Map<String, Object> createOrder(@RequestBody OrderWithItemsDTO orderWithItemsDTO, @RequestHeader("...
2609     logger.info("=== 收到创建订单请求 ===");
2610     logger.info("TenantId: {}", tenantId);
2611
2612     Map<String, Object> response = new HashMap<>();
2613
2614     try {
2615         // 设置租户ID
2616         Order order = orderWithItemsDTO.getOrder();
2617         if (order == null) {
2618             response.put("success", false);
2619             response.put("message", "订单主数据不能为空");
2620             return response;
2621         }
2622         order.setTenantId(tenantId);
2623
2624         // 设置用户ID (避免orders.user_id非空约束失败)
2625         String userIdStr = UserContext.getUserId();
2626         Integer userId = null;
2627         try {
2628             if (userIdStr != null && !userIdStr.isBlank()) {
2629                 userId = Integer.parseInt(userIdStr);
2630             }
2631         } catch (Exception ignored) {
2632             userId = null;
2633         }
2634         if (userId == null) {
2635             response.put("success", false);
2636             response.put("message", "缺少用户上下文");
2637             return response;
2638         }
2639         order.setUserId(userId);
2640
2641         // 创建订单
2642         Order savedOrder = orderService.createOrder(orderWithItemsDTO);
2643
2644         response.put("success", true);
2645         response.put("message", "订单创建成功");
2646         response.put("data", savedOrder);
2647     } catch (Exception e) {
2648         logger.error("创建订单失败: {}", e.getMessage(), e);
2649         response.put("success", false);
2650         response.put("message", "创建订单失败: " + e.getMessage());

```

```

2651     }
2652
2653     return response;
2654 }
2655
2656 /**
2657  * 确认订单（用于AI提取数据）
2658  * 接收包含cust_id、订单主表信息及order_items列表的请求
2659  * 开启事务，写入orders表，批量写入order_items，根据product_id扣减products表的stock
2660  * @param orderWithItemsDTO 订单及详情DTO
2661  * @return 确认结果
2662  */
2663 @PostMapping("/confirm")
2664 public Map<String, Object> confirmOrder(@RequestBody OrderWithItemsDTO orderWithItemsDTO) {
2665     logger.info("=== 收到确认订单请求 ===");
2666
2667     Map<String, Object> response = new HashMap<>();
2668
2669     try {
2670         // 获取租户ID
2671         String tenantId = UserContext.getTenantId();
2672         String userIdStr = UserContext.getUserId();
2673         logger.info("TenantId: {}", tenantId);
2674
2675         // 设置租户ID
2676         Order order = orderWithItemsDTO.getOrder();
2677         if (order == null) {
2678             response.put("success", false);
2679             response.put("message", "订单主数据不能为空");
2680             return response;
2681         }
2682         order.setTenantId(tenantId);
2683         try {
2684             order.setUserId(userIdStr == null ? 0 : Integer.parseInt(userIdStr));
2685         } catch (Exception e) {
2686             order.setUserId(0);
2687         }
2688         if (order.getOrderStatus() == null || order.getOrderStatus().isBlank()) {
2689             order.setOrderStatus("PROCESSING");
2690         }
2691         if (order.getDeliveryDate() == null) {
2692             order.setDeliveryDate(LocalDateTime.now());
2693         }
2694
2695         // 校验cust_id
2696         if (order.getCustId() == null) {
2697             response.put("success", false);
2698             response.put("message", "客户ID不能为空");
2699             return response;
2700         }

```

```

2701
2702 // 校验order_items
2703 List<OrderItem> orderItems = orderWithItemsDTO.getOrderItems();
2704 if (orderItems == null || orderItems.isEmpty()) {
2705     response.put("success", false);
2706     response.put("message", "订单详情不能为空");
2707     return response;
2708 }
2709
2710 // 校验每个order_item的product_id
2711 for (OrderItem item : orderItems) {
2712     if (item.getProductid() == null) {
2713         response.put("success", false);
2714         response.put("message", "产品ID不能为空");
2715         return response;
2716     }
2717     if (item.getItemStatus() == null || item.getItemStatus().isBlank()) {
2718         item.setItemStatus("PENDING");
2719     }
2720     if (item.getDeliveryDate() == null) {
2721         item.setDeliveryDate(order.getDeliveryDate());
2722     }
2723 }
2724
2725 // 创建订单
2726 Order savedOrder = orderService.createOrder(orderWithItemsDTO);
2727
2728 response.put("success", true);
2729 response.put("message", "订单确认成功");
2730 response.put("data", savedOrder);
2731 } catch (Exception e) {
2732     logger.error("确认订单失败: {}", e.getMessage(), e);
2733     response.put("success", false);
2734     response.put("message", "确认订单失败: " + e.getMessage());
2735 }
2736
2737 return response;
2738 }
2739
2740 /**
2741  * 根据订单ID查询订单
2742  * @param orderId 订单ID
2743  * @return 订单信息
2744  */
2745 @GetMapping("/{orderId}")
2746 public Map<String, Object> getOrderById(@PathVariable Integer orderId) {
2747     logger.info("=== 收到查询订单请求 ===");
2748     logger.info("OrderId: {}", orderId);
2749
2750     Map<String, Object> response = new HashMap<>();

```



```

2751
2752     try {
2753         Order order = orderService.getOrderById(orderId);
2754         if (order == null) {
2755             response.put("success", false);
2756             response.put("message", "订单不存在");
2757             return response;
2758         }
2759
2760         response.put("success", true);
2761         response.put("data", order);
2762     } catch (Exception e) {
2763         logger.error("查询订单失败: {}", e.getMessage(), e);
2764         response.put("success", false);
2765         response.put("message", "查询订单失败: " + e.getMessage());
2766     }
2767
2768     return response;
2769 }
2770
2771 /**
2772  * 根据订单编号查询订单
2773  * @param orderCode 订单编号
2774  * @return 订单信息
2775  */
2776 @GetMapping("/code/{orderCode}")
2777 public Map<String, Object> getOrderByOrderCode(@PathVariable String orderCode) {
2778     logger.info("=== 收到查询订单请求 ===");
2779     logger.info("OrderCode: {}", orderCode);
2780
2781     Map<String, Object> response = new HashMap<>();
2782
2783     try {
2784         Order order = orderService.getOrderByOrderCode(orderCode);
2785         if (order == null) {
2786             response.put("success", false);
2787             response.put("message", "订单不存在");
2788             return response;
2789         }
2790
2791         response.put("success", true);
2792         response.put("data", order);
2793     } catch (Exception e) {
2794         logger.error("查询订单失败: {}", e.getMessage(), e);
2795         response.put("success", false);
2796         response.put("message", "查询订单失败: " + e.getMessage());
2797     }
2798
2799     return response;
2800 }

```

```

2801
2802 /**
2803  * 根据租户ID查询订单列表
2804  * @param tenantId 租户ID
2805  * @return 订单列表
2806  */
2807 @GetMapping
2808 public Map<String, Object> getOrdersByTenantId(@RequestHeader("X-Tenant-Id") String tenantId) {
2809     logger.info("=== 收到查询订单列表请求 ===");
2810     logger.info("TenantId: {}", tenantId);
2811
2812     Map<String, Object> response = new HashMap<>();
2813
2814     try {
2815         List<Order> orders = orderService.getOrdersByTenantId(tenantId);
2816         response.put("success", true);
2817         response.put("data", orders);
2818     } catch (Exception e) {
2819         logger.error("查询订单列表失败: {}", e.getMessage(), e);
2820         response.put("success", false);
2821         response.put("message", "查询订单列表失败: " + e.getMessage());
2822     }
2823
2824     return response;
2825 }
2826
2827 /**
2828  * 查询进行中订单（待支付/待出货）
2829  * 状态编码: D010001, D010002
2830  */
2831 @GetMapping("/in-progress")
2832 public Map<String, Object> getInProgressOrders(@RequestHeader("X-Tenant-Id") String tenantId) {
2833     logger.info("=== 收到查询进行中订单请求 ===");
2834     logger.info("TenantId: {}", tenantId);
2835
2836     Map<String, Object> response = new HashMap<>();
2837
2838     try {
2839         List<String> statuses = Arrays.asList("D010001", "D010002");
2840         List<Order> orders = orderService.getOrdersByTenantIdAndStatuses(tenantId, statuses);
2841         response.put("success", true);
2842         response.put("data", orders);
2843     } catch (Exception e) {
2844         logger.error("查询进行中订单失败: {}", e.getMessage(), e);
2845         response.put("success", false);
2846         response.put("message", "查询进行中订单失败: " + e.getMessage());
2847     }
2848
2849     return response;
2850 }

```

```

2851 * 根据租户ID查询最新的10条订单
2852 * @param tenantId 租户ID
2853 * @return 订单列表
2854 */
2855 @GetMapping("/latest")
2856 public Map<String, Object> getLatestOrdersByTenantId(@RequestHeader("X-Tenant-Id") String tenantId) {
2857     logger.info("=== 收到查询最新10条订单请求 ===");
2858     logger.info("TenantId: {}", tenantId);
2859
2860     Map<String, Object> response = new HashMap<>();
2861
2862     try {
2863         List<Order> orders = orderService.getLatestOrdersByTenantId(tenantId);
2864         response.put("success", true);
2865         response.put("data", orders);
2866     } catch (Exception e) {
2867         logger.error("查询最新订单列表失败: {}", e.getMessage(), e);
2868         response.put("success", false);
2869         response.put("message", "查询最新订单列表失败: " + e.getMessage());
2870     }
2871
2872     return response;
2873 }
2874
2875 /**
2876 * 更新订单状态
2877 * @param orderId 订单ID
2878 * @param status 新的订单状态
2879 * @return 更新结果
2880 */
2881 @PutMapping("/{orderId}/status")
2882 public Map<String, Object> updateOrderStatus(@PathVariable Integer orderId, @RequestParam String status) {
2883     logger.info("=== 收到更新订单状态请求 ===");
2884     logger.info("OrderId: {}, Status: {}", orderId, status);
2885
2886     Map<String, Object> response = new HashMap<>();
2887
2888     try {
2889         Order updatedOrder = orderService.updateOrderStatus(orderId, status);
2890         if (updatedOrder == null) {
2891             response.put("success", false);
2892             response.put("message", "订单不存在");
2893             return response;
2894         }
2895
2896         response.put("success", true);
2897         response.put("message", "订单状态更新成功");
2898         response.put("data", updatedOrder);
2899     } catch (Exception e) {
2900         logger.error("更新订单状态失败: {}", e.getMessage(), e);

```

```

2901         response.put("success", false);
2902         response.put("message", "更新订单状态失败: " + e.getMessage());
2903     }
2904
2905     return response;
2906 }
2907
2908 /**
2909  * 根据订单ID查询订单详情
2910  * @param orderId 订单ID
2911  * @return 订单详情列表
2912  */
2913 @GetMapping("/{orderId}/items")
2914 public Map<String, Object> getOrderItemsByOrderId(@PathVariable Integer orderId) {
2915     logger.info("=== 收到查询订单详情请求 ===");
2916     logger.info("OrderId: {}", orderId);
2917
2918     Map<String, Object> response = new HashMap<>();
2919
2920     try {
2921         List<?> orderItems = orderService.getOrderItemsByOrderId(orderId);
2922         response.put("success", true);
2923         response.put("data", orderItems);
2924     } catch (Exception e) {
2925         logger.error("查询订单详情失败: {}", e.getMessage(), e);
2926         response.put("success", false);
2927         response.put("message", "查询订单详情失败: " + e.getMessage());
2928     }
2929
2930     return response;
2931 }
2932
2933 /**
2934  * 根据客户ID查询订单
2935  * @param custId 客户ID
2936  * @return 订单列表
2937  */
2938 @GetMapping("/customer/{custId}")
2939 public Map<String, Object> getOrdersByCustomerId(@PathVariable Integer custId) {
2940     logger.info("=== 收到根据客户ID查询订单请求 ===");
2941     logger.info("CustId: {}", custId);
2942
2943     Map<String, Object> response = new HashMap<>();
2944
2945     try {
2946         List<Order> orders = orderService.getOrdersByCustId(custId);
2947         response.put("success", true);
2948         response.put("data", orders);
2949     } catch (Exception e) {
2950         logger.error("查询客户订单失败: {}", e.getMessage(), e);

```

```

2951         response.put("success", false);
2952         response.put("message", "查询客户订单失败: " + e.getMessage());
2953     }
2954
2955     return response;
2956 }
2957
2958 /**
2959  * 时间窗内指定产品的销售件数合计（进货建议生成用，quantity 为基本单位件数）
2960  */
2961 @PostMapping("/sales-sum-by-products")
2962 public Map<String, Object> salesSumByProducts(@RequestBody Map<String, Object> body) {
2963     Map<String, Object> response = new HashMap<>();
2964     try {
2965         String tenantId = UserContext.getTenantId();
2966         if (tenantId == null || tenantId.isBlank()) {
2967             response.put("success", false);
2968             response.put("message", "缺少租户上下文");
2969             return response;
2970         }
2971         String fromStr = String.valueOf(body.getOrDefault("from", ""));
2972         String toStr = String.valueOf(body.getOrDefault("to", ""));
2973         LocalDateTime from = LocalDateTime.parse(fromStr);
2974         LocalDateTime to = LocalDateTime.parse(toStr);
2975         @SuppressWarnings("unchecked")
2976         List<Object> rawIds = (List<Object>) body.get("productIds");
2977         if (rawIds == null || rawIds.isEmpty()) {
2978             response.put("success", true);
2979             response.put("data", Map.of());
2980             return response;
2981         }
2982         List<Integer> productIds = new ArrayList<>();
2983         for (Object o : rawIds) {
2984             productIds.add(((Number) o).intValue());
2985         }
2986         Map<Integer, Long> sums = orderService.sumSoldQuantitiesForProducts(tenantId, from, to, productIds);
2987         Map<String, Object> out = new HashMap<>();
2988         for (Map.Entry<Integer, Long> e : sums.entrySet()) {
2989             out.put(String.valueOf(e.getKey()), e.getValue());
2990         }
2991         response.put("success", true);
2992         response.put("data", out);
2993     } catch (Exception e) {
2994         logger.error("销售汇总失败: {}", e.getMessage(), e);
2995         response.put("success", false);
2996         response.put("message", "销售汇总失败: " + e.getMessage());
2997     }
2998     return response;
2999 }
3000 }

```